

IT-teknolog

Optilogic rapport



Forfattere

Sebastian, Magnus, Daniel & Lex

Onsdag den 3. dec. 2025

Indholdsfortegnelse

Indholdsfortegnelse	2
Introduktion.....	1
Problemformulering	1
Afgrænsning.....	1
Teoretisk grundlag.....	2
Komponentliste.....	2
Udviklingsprocessen - Kravanalyse og arkitekturdesign	2
Kravspecifikationer	2
Funktionelle krav	2
Furps modellen:	3
Arkitekturdesign	4
Integreret Arkitektur for Edge Computing, Datakommunikation og Systemstyring	6
Edge-computing & dataopsamling	6
Datakommunikation & standardisering.....	7
Systemarkitektur & containerisering	7
Datahåndtering og applikationslogik	7
Systemstyring og adgangskontrol	8
Kodesprog og struktur	8
Netværkstopologi.....	9
Metodologi.....	9
Klassediagram/Funktionsdiagram	10
Industrielle protokoller.....	11
Modbus RTU	11
Modbus TCP/IP	11
SPARKPLUG B	12
HTTP.....	12
Resultater & Analyse	12
Prototype	12
Funktionaliteter	13
Docker compose	13

Server funktionalitet	13
Frontend	13
Logging	15
API.....	15
MQTT	16
Healtchecks	17
QuestDB	18
Emulator/Ingestor funktionalitet.....	19
Emulatoren	19
Ingestor	20
Deployment	21
ESP32 – Olimex funktionalitet.....	22
MQTT - Olimex	23
Unit-Test resultater	25
Diskussion	26
Perspektivering	26
Bilag	28

Introduktion

Optilogics nuværende løsninger indeholder systemer og komponenter, som ikke selv har mulighed for at registrere eller logge deres data. Dette betyder, at værdifuld viden om systemernes driftstilstand går tab og at der mangler indsigt i systemernes data over tid. Uden adgang til disse data bliver det vanskeligt at identificere tendenser, fejl og optimeringsmuligheder.

Optilogics medarbejdere og fagpersoner har den nødvendige ekspertise til at optimere systemernes performance, men deres arbejde begrænses af manglen på konkret driftsdata. Uden et system, der indsamler og synliggør data, kan det være udfordrende både at dokumentere systemadfærd og træffe beslutninger baseret på fakta fremfor antagelser.

Projektets formål er at indsamle data fra en ventilator, dette skal indhentes gennem en ESP-32 Olimex, hvor hele server-løsningen afvikles ved brug af docker-containere.

I samarbejde med Optilogic er der identificeret programmer, der er egnede i forhold til opsætning, administration og vedligeholdelse. Der anvendes QuestDB, Grafana, FastAPI, Flask og MQTT. Systemet afvikles på et Zimaboard med CasaOS, hvor alle containere administreres gennem Portainer for at sikre enkel administration og effektiv fejlfinding.

Problemformulering

Den centrale udfordring består i at udvikle en robust, skalerbar og sikker arkitektur, der muliggør opsamling, transmission, lagring og præsentation af driftsdata fra et eksisterende system, som ikke selv understøtter datalogning.

Hvordan kan vi designe og implementere en end-to-end datainfrastruktur der muliggør modtagelse, overblik og lagring med brugertilladelser.

Afgrænsning

Projektet omfatter ikke udvikling af Machine-learning analyseteknikker eller automatiseret beslutningstagning. Fokus er udelukkende på dataindsamling, validering, persistens og visualisering. ML-analyse anses som et potentielt videre forskningsspor, der kan anvende den etablerede datainfrastruktur som fundament for fremtidig udvikling og optimering.

TLS/SSL er ikke omfattet i projektet på dette tidspunkt. Fokus for dette projekt er udelukkende at få data videresendt, opbevaret og præsenteret. TLS/SSL vil være blandt det næste logiske skridt i projektet for at tilføje noget sikkerhed til kommunikationen mellem enhederne.

Teoretisk grundlag

Dette projekt bygger på teoretiske principper indenfor Indlejrede systemer, Programmering, Netværk struktur og moderne softwarearkitektur.

Komponentliste

- Zimaboard – Linux server
- ESP32 – Olimex
- Mod-RS485
- 3D Print – Case
- Router - Edimax

Udviklingsprocessen - Kravanalyse og arkitekturdesign

For at imødekomme idégenereringen og prototype udvikling, er der lavet en kravanalyse og et arkitekturdesign. Disse er med til at skabe overblik over projektets formål og udviklings dimension.

Kravspecifikationer

Funktionelle krav

Systemet skal:

- Udsende MQTT-beskeder.
- Publicere DBIRTH, DDATA og DDEATH.
- Modtage rå Modbus data eller simuleret data via emulator.
- Sende struktureret payload videre via MQTT.
- FastAPI skal give adgang til data via REST-endpoints.
- Portainer udføre healthchecks for backend.
- Frontend skal vise grafer med live-data.
- Healthchecks skal kunne ses på frontend.
- Indsætte alle relevante datapunkter i QuestDB, inkl. valid datamodel og korrekt timestamping.
- Køre hele systemet i Docker Compose som en samlet løsning uden init-fejl eller container-looping.
- Håndtere brugergodkendelse i Flask-frontenden:
 - Blokere bypass-forsøg mod loginflow.
 - Afvise ugyldige credentials kontrolleret og reproducerbart.

Ikke-funktionelle krav

Systemet skal:

- Semi-automatisk generere dokumentation (Doxygen)
- kunne køre stabilt under belastning (MQTT + QuestDB)
- Have testdækning for login-flowet, inkl. edge cases.
- Sikre reproducerbar opsætning via docker-compose.

Functionality

Systemets funktionelle kvalitetskrav omfatter sikker datahåndtering, standardiseret kommunikation og kontrolleret adgang.

- Sparkplug-B anvendes som fælles dataprotokol mellem edge-device, MQTT og backend.
- DBIRTH, DDATA og DDEATH publiceres korrekt fra edge-enheden.
- Kun autoriserede brugere har adgang til dashboard og administrationsfunktionerne.
- Systemet logger centrale hændelser (Fejl, Info, warnings og Healthchecks)
- FastAPI som datalag mellem QuestDB og frontend.
- QuestDB-insert + query-path testet for stabilitet.
- Dashboard med live-data.

Usability

Systemet skal være let at anvende og overskueligt.

- Dashboard viser temperatur, tryk og airflow i en live-data graf.
- Brugere kan bruge siden uden teknisk viden.
- Login og navigation på siden holdes intuitivt.
- Health status synlig direkte i dashboard.
- FastAPI giver ensartede JSON-responses til videre behandling.

Reliability

Systemet skal levere stabil drift, kontinuerlige data og korrekt fejlhåndtering.

- Last Will and Testament og DDEATH sikrer korrekt offline-status i MQTT-broker.
- Docker Compose opstart i korrekt sekvens og fejlfrit.
- Stabil datatilførsel til QuestDB uden tab af samples.
- Healthchecks returnerer korrekt systemtilstand.
- Unittests sikrer stabilt loginflow.

Performance

Systemet skal kunne levere data og visualisering med lav responstid.

- Indsættelser i QuestDB kører uden blocking.
- FastAPI-endpoints responderer uden unødvendig ventetid.
- Dashboard opdateres med lav respons tid ved nye datapunkter.
- Docker-miljøet skal håndtere services parallelt uden forsinkelse.

Supportability

Systemet skal kunne vedligeholdes, dokumenteres og genskabes uden komplekse manuelle skridt.

- Dokumentation generet i Doxygen.
- Klar mappestruktur og brug af best practices i Docker Compose.
- Projektet kan genskabes via Docker Compose.
- Emulatoren muliggør fejlfinding og udvikling uden fysisk ESP32.

Arkitekturdesign

Nedenfor ses det oprindelige og nye udviklings flowchart der blev bygget som projektets grundlæggende byggesten. Det var vigtigt for projektet at der inden udvikling var et fundamental overblik over systemtopologien og hvordan flowet i de forskellige systemer skulle fungere.

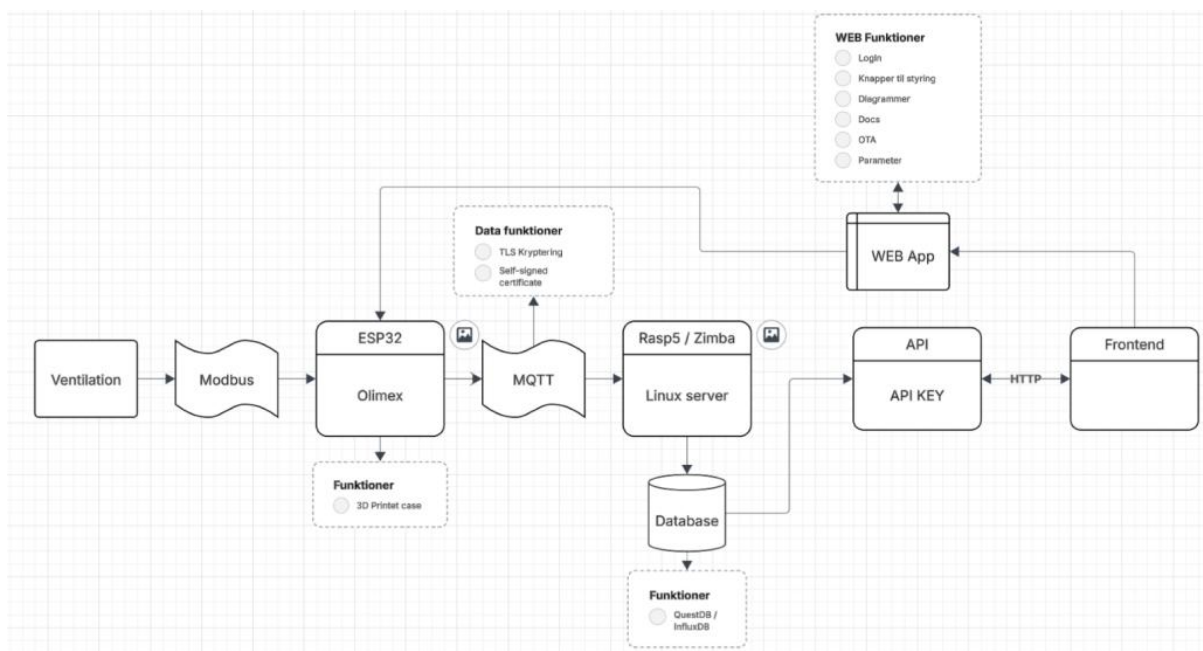
Flowet begyndte ved selve ventilationsanlægget, hvor data blev sendt via modbus. Denne del markerede tydeligt, at systemet skulle kunne indsamle rå målinger direkte fra eksisterende industriel infrastruktur. Det var altså essentielt at vores edge-device kunne modtage, forstå og parse modbus data.

Derudover var det også nødvendigt at den data der blev sent forblev en industri standard, *SparkPlug B* standarden. Dette fungerede som den primære kommunikationskanal mellem edge-enheden og servermiljøet, hvor overgangen til Linux serveren gjorde det muligt at opbevarer vores data i Questdb.

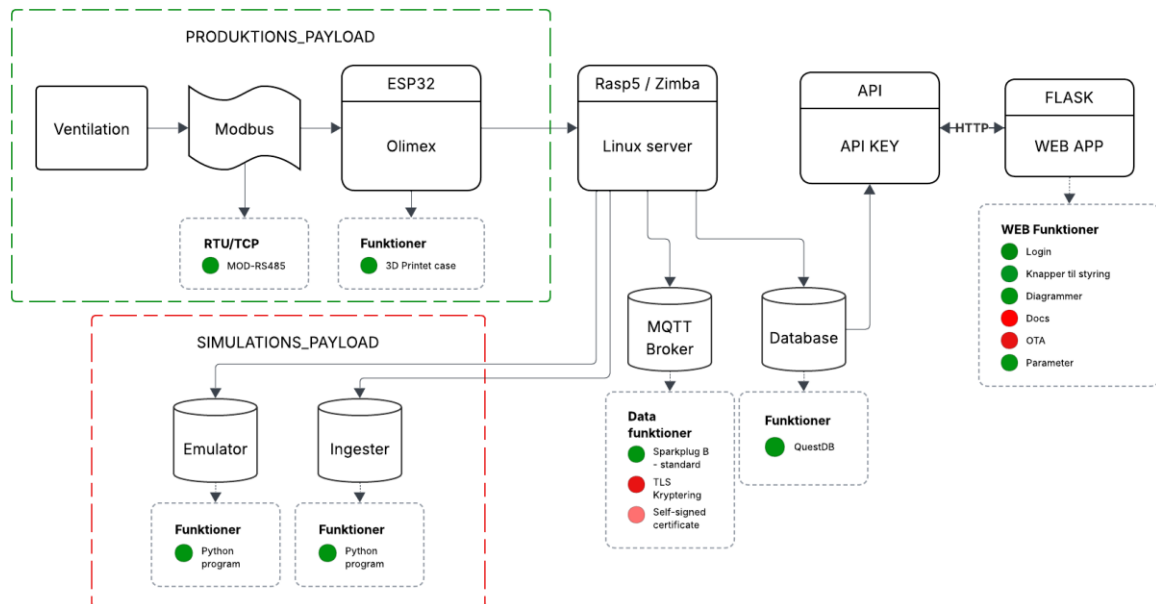
På serversiden blev der i første iteration også tænkt ind, at API'en skulle kunne håndtere forespørgsler fra webapplikationen. Formålet var at skabe et fleksibelt bindeled, som kunne håndtere både datalagring, hentehistorik og styringsfunktioner.

Frontend-delen blev visualiseret som den sidste systemdel i kæden. Her skulle der være en skræddersyet bruger opdelt interface, som skulle have funktioner som login, visning af grafer, styringsknapper, opdatering af parametre samt healthcheck.

Det var netop dette overblik der gjorde det muligt at videreudvikle på projektet i senere iterationer og raffinere både datastruktur, server-workflow, healthchecks, unit-testing, samt frontend-funktioner.

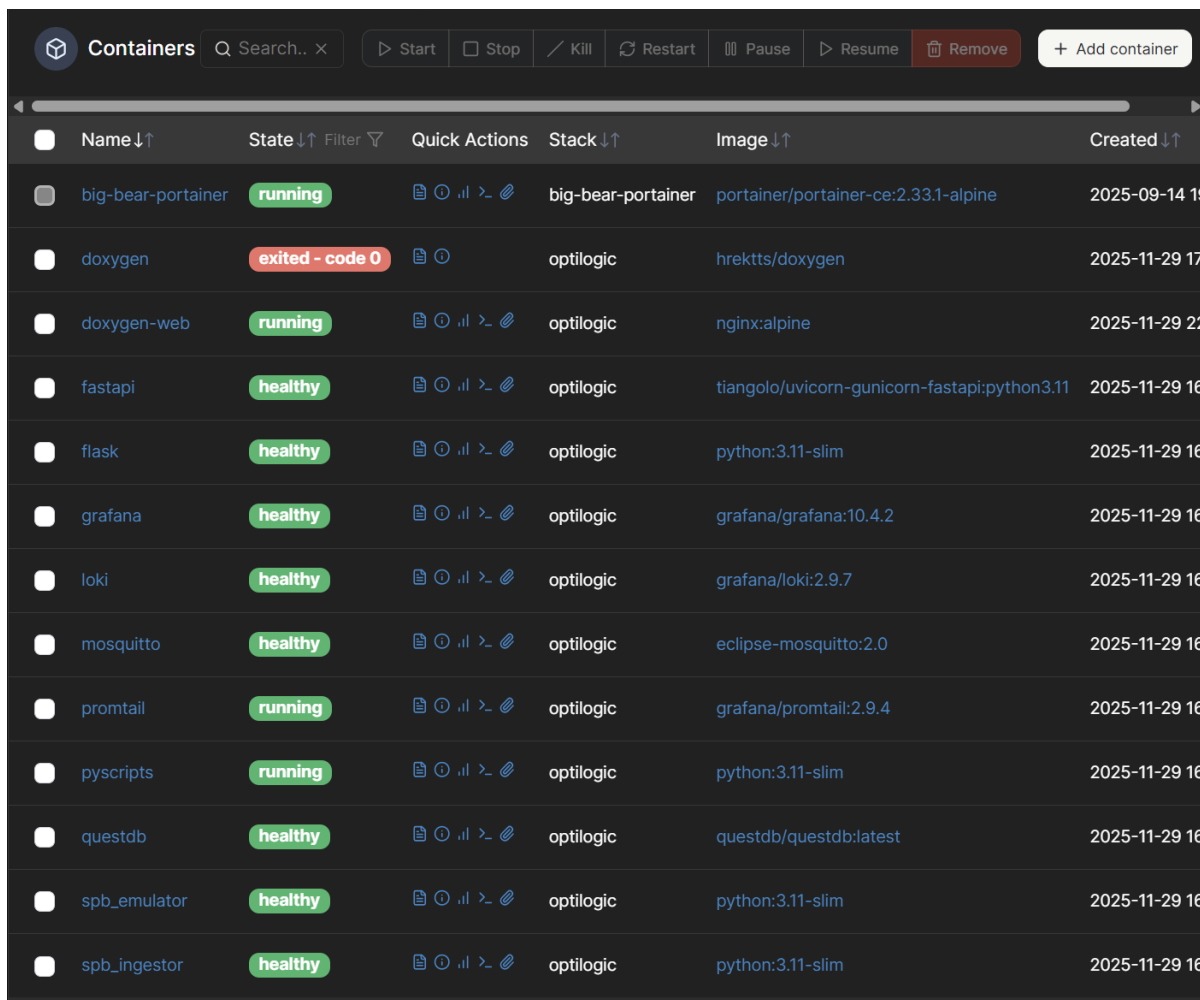


Gamle arkitektur



Nye arkitektur

Integreret Arkitektur for Edge Computing, Datakommunikation og Systemstyring



The screenshot shows a Docker container management interface. At the top, there's a header with the 'Containers' title, a search bar, and several action buttons: Start, Stop, Kill, Restart, Pause, Resume, Remove, and Add container. Below the header is a table listing various containers. Each row includes a checkbox, the container name, its state (e.g., running, exited, healthy), quick actions (stop, restart, kill, pause, resume), the stack name, the image name, and the creation time.

	Name ↓↑	State ↓↑ Filter ▾	Quick Actions	Stack ↓↑	Image ↓↑	Created ↓↑
☐	big-bear-portainer	running		big-bear-portainer	portainer/portainer-ce:2.33.1-alpine	2025-09-14 19
☐	doxygen	exited - code 0		optilogic	hrekts/doxygen	2025-11-29 17:
☐	doxygen-web	running		optilogic	nginx:alpine	2025-11-29 22
☐	fastapi	healthy		optilogic	tiangolo/uvicorn-gunicorn-fastapi:python3.11	2025-11-29 16
☐	flask	healthy		optilogic	python:3.11-slim	2025-11-29 16
☐	grafana	healthy		optilogic	grafana/grafana:10.4.2	2025-11-29 16
☐	loki	healthy		optilogic	grafana/loki:2.9.7	2025-11-29 16
☐	mosquitto	healthy		optilogic	eclipse-mosquitto:2.0	2025-11-29 16
☐	promtail	running		optilogic	grafana/promtail:2.9.4	2025-11-29 16
☐	pyscripts	running		optilogic	python:3.11-slim	2025-11-29 16
☐	questdb	healthy		optilogic	questdb/questdb:latest	2025-11-29 16
☐	spb_emulator	healthy		optilogic	python:3.11-slim	2025-11-29 16
☐	spb_ingestor	healthy		optilogic	python:3.11-slim	2025-11-29 16

Den overordnet struktur i den samlede drift af projektets portainer backendmiljø er systemets mikrotjenester, herunder FastAPI, Flask, MQTT-broker og database. Grafana, Promtail, Loki samt databehandlings- og edge-emulator-services, kører isoleret i hver sin Docker-container. Denne struktur danner fundamentet for projektets end-to-end platform og sikrer stabil, skalerbar og reproducerbar drift på tværs af alle systemkomponenter.

Edge-computing & dataopsamling

Projektets dataindsamling baseres på ESP32 Olimex samt modulet (MOD-RS485), som fungerer som edge-enheder. Disse enheder kommunikerer via Modbus-protokollen, et udbredt industrielt standardformat til overførsel af måledata fra ventilationsmodulet.

Edge computing-princippet understøtter lokal databehandling og reducerer belastningen på netværket ved kun at sende relevante data videre til backenden. Denne tilgang bygger på teorien om distribueret databehandling, hvor intelligens flyttes ud mod datakilden for at øge robusthed og effektivitet.

Datakommunikation & standardisering

Til datatransmission anvendes MQTT-protokollen, som bygger på *publish/subscribe*-arkitekturen. Dette sikrer asynkron kommunikation, lav båndbredde og høj pålidelighed i ustabile netværksmiljøer.

Systemet implementerer samtidig Sparkplug B-standarden, der udvider MQTT med en fælles datamodel, så alle komponenter forstår hinanden. Denne standard understøtter *state awareness*, *node management* og en struktureret identifikation af datakilder, hvilket giver en mere robust og harmonisk kommunikationsinfrastruktur.

Grunden til at industristandarden Sparkplug B er blevet valgt til dette projekt, er fordi Sparkplug B sørger for, at alle enheder i systemet sender deres data på en ens måde. Det betyder, at backend altid ved præcis, hvor dataene kommer fra, hvad de betyder og om enhederne er online eller offline. Standarden gør det nemmere at holde styr på hver enkelt måling, fordi både enheder og datapunkter har faste og tydelige navne. Det hjælper også systemet med hurtigt at opdage, hvis en enhed mister forbindelsen eller genstarter.

Overordnet gør det hele platformen mere robust, reducerer kompleksiteten i databehandlingen og giver et mere pålideligt grundlag for både monitorering, fejlhåndtering og udvidelse af systemet.

Systemarkitektur & containerisering

Backenden er opbygget som en mikrotjenestearkitektur, hvor komponenter som MQTT-broker, database, API og frontend kører i isolerede Docker-containere, defineret i en Compose-fil.

Containerisering bygger på teorien om software isolation og reproducérbarhed, som muliggør ensartet drift, enkel vedligeholdelse og horisontal skalering. Denne arkitektur tillader løbende udvikling og opdatering uden nedetid.

Datahåndtering og applikationslogik

Systemet anvender QuestDB som database, der er optimeret til hurtig behandling af sekventielle målinger. Den kolonnebaserede struktur og tidsindeksering muliggør effektiv forespørgsel og analyse over store datamængder.

Der findes to hovedformer for datastrukturer indenfor SQL-databaser, Struktureret SQL og ikke struktureret SQL. Kendetegn er fast-tabelstruktur som går ind under den struktureret form. Filer der kræver tolkning, som f.eks. video/billeder, PDF og tekst er ustruktureret. Dette resulterede i brugen af den strukturerede tabelstruktur i Optilogics database, da der bliver arbejdet med sensor værdier og målinger. Udover det, skaber tabelfunktionen et bedre overblik samt nem tilgang til data

Mellem databasen og præsentationslaget anvendes FastAPI som Application Programming Interface, hvilket understøtter asynkrone forespørgsler og REST-baseret kommunikation.

Frontenden er udviklet i Flask, som håndterer brugerinteraktion, datafremvisning og systemstyring. Denne todeling mellem API og frontend afspejler teorien om “separation of concerns”, hvor logik, data og præsentation adskilles for at øge fleksibilitet og vedligeholdelse.

Brugerfladen er bygget i HTML, CSS og javascript. Flask leverer data fra API'en til frontenden. Denne struktur giver en klar opdeling mellem logikken i Python og præsentationen i HTML. HTML-siderne holdes enkle og lette at loade, mens Python står for datahentning, endpoint-kald og sikkerhed gennem API. API'en udfører forespørgslen mod backend, så klienten ikke får direkte adgang til interne systemer.

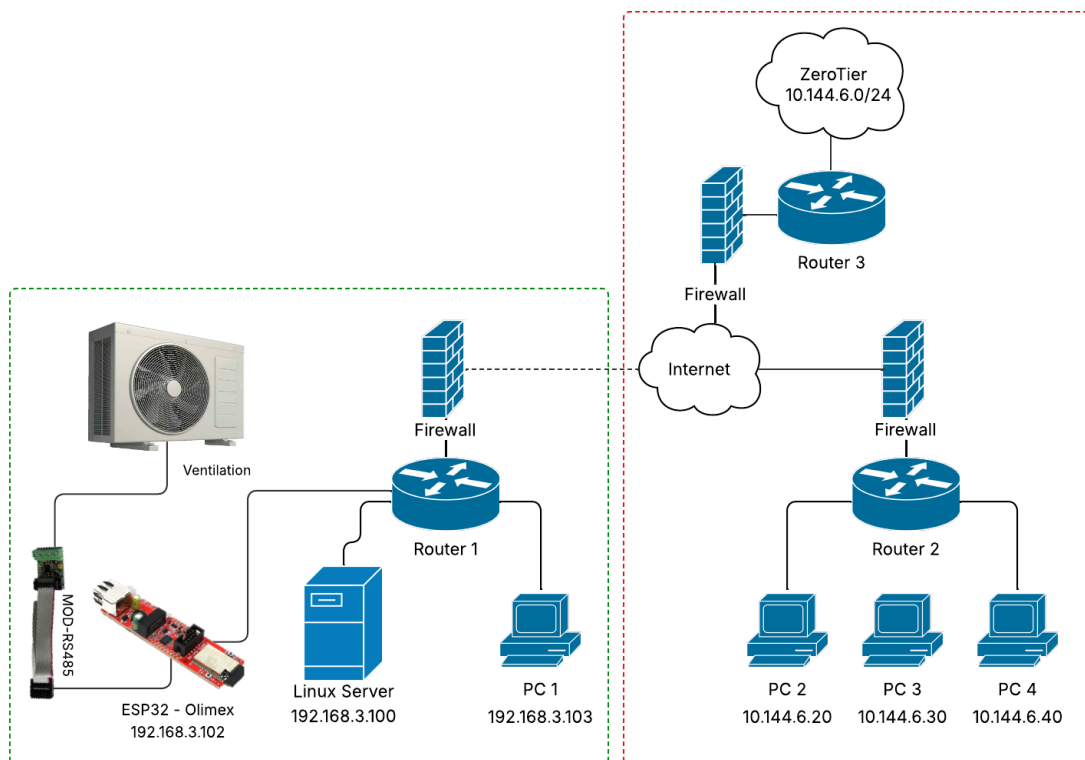
Systemstyring og adgangskontrol

Der anvendes Role-Based Access Control (RBAC) til håndtering af brugerrettigheder, så forskellige brugere kun har adgang til de noder og funktioner, de er autoriseret til. Dette understøtter informationssikkerhed og systemintegritet.

Kodesprog og struktur

C++ anvendes på Olimex enheden for at sikre en høj ydeevne og effektiv MQTT-håndtering. Flask, FastAPI, QuestDB integration samt Grafana opsætningen, er implementeret i Python. C++ kunne have været anvendt til FastAPI delen for at opnå mere stabil kommunikation og højere ydeevne i miljøer med større datamængder.

Frontenden er implementeret i Python, men selve siden er opsat i HTML.



Metodologi

Edge-enheden består af ESP32 med Modbus-modul, som indsamler rå måleværdier fra ventilatormodulet og omsætter dem til et struktureret payloadformat, der kan sendes videre.

I perioder hvor der ikke har været mulighed for faktisk data, er der gjort brug af emuleret data. Dette har resulteret i udsendelse af syntetiske DBIRTH og DDATA-meddelelser i Sparkplug-B format.

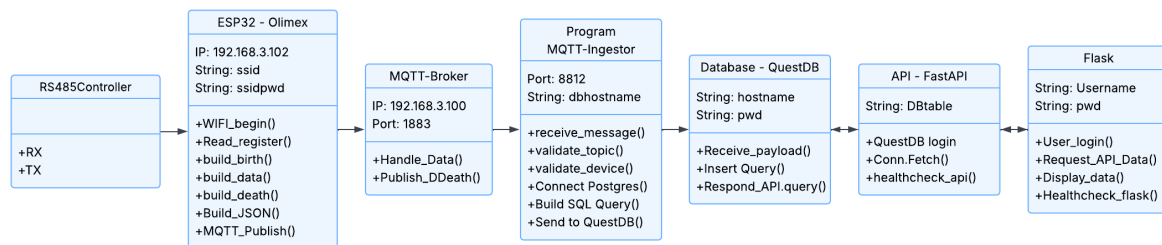
Kommunikations protokollen er MQTT, som distribuerer payloaden uændret fra edge-enheden eller emulatoren til backendens abonnenter. Ingestoren håndtere efterfølgende aflæsning af disse meddelelser, kontrollerer protokoloverholdelse og konverterer indholdet til et format, der kan indsættes i databasen.

Databasen er QuestDB, som lagrer alle tidsseriedata og gøres tilgængelig som strukturerede query-resultater. FastAPI levere kontrolleret adgang til data og returnere JSON-baserede svar til de komponenter, der efterspørger information. Frontenden er bygget i Flask og præsenterer målinger i grafer som giver mulighed for observering af systemstatus.

Datavalidering: Systemet anvender faste kontrolpunkter, hvor de kontrollerede payload, topic-struktur og datatyper valideres, før de sendes videre. Det er kun beskeder der opfylder format og payload kravene, der bliver behandlet.

Standardisering: Alle rå værdier konverteres til et ensartet datasæt, så MQTT, ingestoren og databasen modtager i samme struktur uanset om de kommer fra en fysisk enhed eller en emulator.

Klassediagram/Funktionsdiagram



Dette funktions-/klassediagram giver et samlet overblik over hele systemets dataflow og komponentansvar – fra indsamling af rå mobus-data på en ESP32-enhed til lagring, API-udstilling og visning i et Flask-baseret dashboard. Diagrammet beskriver de centrale moduler, samt viser de vigtigste attributter (såsom IP-adresser, porte og credentials). Derudover fremhæves de funktioner, hver komponent udfører. Samtidig visualiserer det, hvordan data bevæger sig gennem systemet via MQTT, behandles af en dedikeret ingestor, lagres i QuestDB og til sidst hentes af en FastAPI-service, som Flask-klienten anvender.

ESP32 - Olimex repræsenterer den fysiske enhed, der indsamler data fra ventilatoren og bygger en struktureret payload, samt sender den via MQTT.

MQTT-broker indgår som en ekstern transportkomponent, der distribuerer beskeder uden yderligere behandling.

MQTT-ingestor står for indskrivning af de godkendte datapunkter i QuestDB.

API - FastAPI udgør systemets API-lag, hvor både dataudlevering, adgangskontrol og generelle helbredstjek af API-funktionaliteten administreres.

Flask/frontend fungerer som præsentationslag, hvor API-data indhentes og vises for brugeren.

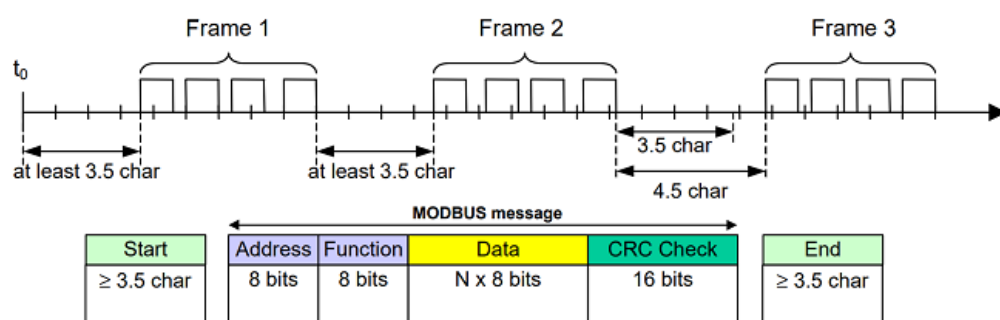
Tilsammen viser klassediagrammet en opdeling, hvor hver klasse har et afgrænset teknisk ansvar og indgår som led i en samlet datakæde fra end to end, altså fra måling til visualisering.

Industrielle protokoller

Modbus RTU

MODBUS RTU er en kommunikations protokol der tillader kommunikation på en delt informationskanal. Modbus er bygget på et master/slave koncept, hvor det altid masteren der starter kommunikation, og slaven der responderer på beskeden.

En modbus besked følger et struktureret format, der er “adresse, funktion, data felt, og checksum”. Det fungerer ved masteren starter kommunikation med en request, den fortæller via beskeden, hvilken start adresse og hvilke registre der skal aflæses, slaven svare. Afsendelse af beskeder foregår over serielle interfaces som RS-232 eller RS-485.



En Modbus frame er opbygget som vist i ovenstående billede. Adressen er til slaven og fortæller hvilken slave der skal svare. Function fortæller den action der skal foretages, om det er en read eller write operation. Data er den besked der skal sendes, f.eks. i form af tal eller kommandoer der skal læses fra slaven eller skrives til slaven. CRC-check kontrollerer for fejl og verificere dataen.

Der er forskel på RS-232 og RS-485. RS-485 er mere omfattende og har flere egenskaber, samt mere bredbånd til datatransmission. RS-232 virker kun optimalt over 15 meter hvor RS-485 virker optimalt op til 1200 meter, samt kan overføre mere data ad gangen. RS-485 har også bedre immunitet overfor elektrisk støj, da dens operating voltage er højere end RS-232, da det kræver mere strøm at sende mere data ad gangen. Mere strøm giver et større magnetfelt rundt om forbindelsen, dette betegnes som støj immunitet.

Modbus TCP/IP

Modbus TCP/IP er en kommunikations protokol der kører server/client princippet, til at forbinde mellem PLC'er, elektroniske enheder og andre styringsformer. Dataen er præsenteret ligesom i modbus RTU, men der bliver brugt et Ethernet interface til kommunikationen. Formålet med TCP er at sikre, at alle pakker bliver sendt og modtaget intakte. IP bliver brugt til at adressere pakker korrekt og route dem til de korrekte interfaces. TCP/IP er kun en transport protokol, og ændrer ikke fortolkning af datastrukturen, det er stadig MODBUS data der bliver sendt.

SPARKPLUG B

Sparkplug B er en industriel standard der omhandler brugen og opsætningen af MQTT-datatransmission.

Der er en standard for opsætning af Topic, strukturen af MQTT-payloads, og typen af payloads. Topic skal opsættes sådan det klart og tydeligt at tyde hvor dataen kommer fra f.eks.

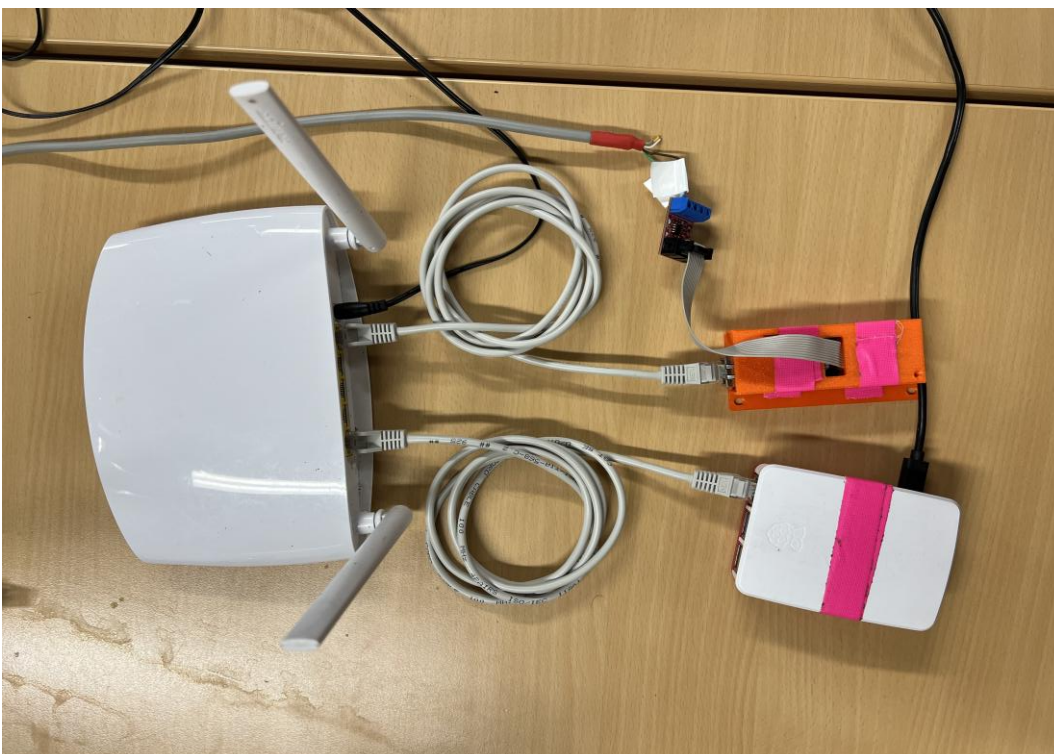
“PlantA/buildingB/room4/senstemp” her er det nemt at tyde området (PlantA), bygningen, lokalet og hvad infoen omhandler. Strukturen af beskeden er i form af at man enten sender en DDATA, DBIRTH, eller DDEATH. DDATA er det kontinuerlige flow af data, gennem MQTT. DBIRTH er på initial connection og fortæller edge-devicet at det nu er kommet online. DDEATH er sendt sådan at brokeren kan fortælle klienter på subscriber siden, at edge-devicet er gået ned, her følger der ofte et Last will and testament med. Last will and testament er en besked type der bliver sendt og retained af MQTT-brokeren, med information om edge-devicet er gået ned.

HTTP

HTTP, hyper text transfer protocol er et applikationslag som i dette projekt bliver brugt til at kommunikere mellem webbrowsers og webservere, samt til API kommunikation. HTTP følger ligesom TCP client/server modellen, klienten starter forbindelsen, og laver en request. HTTP har også GET og POST funktioner til overførsel af data til og fra servers. Der findes også HTTPS, det er den sikre udgave af HTTP og er krypteret med SSL/TLS-certifikater, dette er ikke taget i brug i dette projektet.

Resultater & Analyse

Prototype



Prototypen i ovenstående billede overholder funktionskravene der er blevet stillet til projektet. Til implementering i et produktionsmiljø kræves der videreudvikling på services og funktioner på systemet.

Funktionaliteter

Docker compose

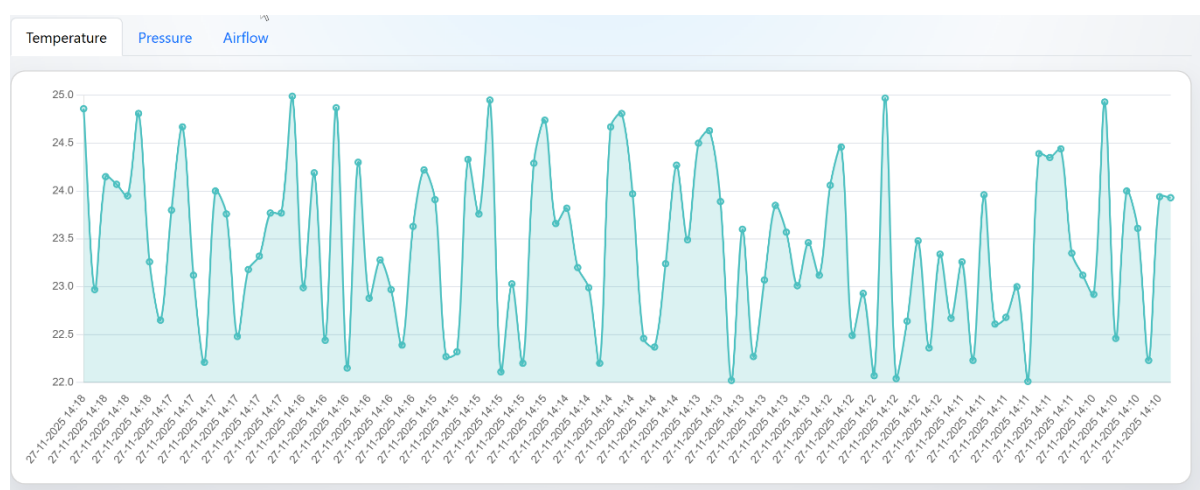
Filen omtalt i dette stykke kan findes i bilag 1.

Docker compose er blevet brugt til at opsætte alle containere i en fil, hvilket tilbyder en nem mappe struktur og godt overblik. I filen kan det ses at containere bliver opsat med de nødvendige indstillinger og variabler. Port mapping i portainer gør det nemt at aflæse porte til de relevante GUI's i browseren. Man kan i composefilen se den mappe struktur der bliver brugt til opsætning af diverse koder og konfigurationer til containere. Mappedstrukturen bliver mappet inde i containeren. Koder og brugernavne vist i compose filen er til interne forbindelser, der ikke kan tilgås udefra. Koden og brugernavn under grafana er kun til første gangs opsætning, disse bliver ændret ved konfigurerings af Dashboardet i browseren.

Server funktionalitet

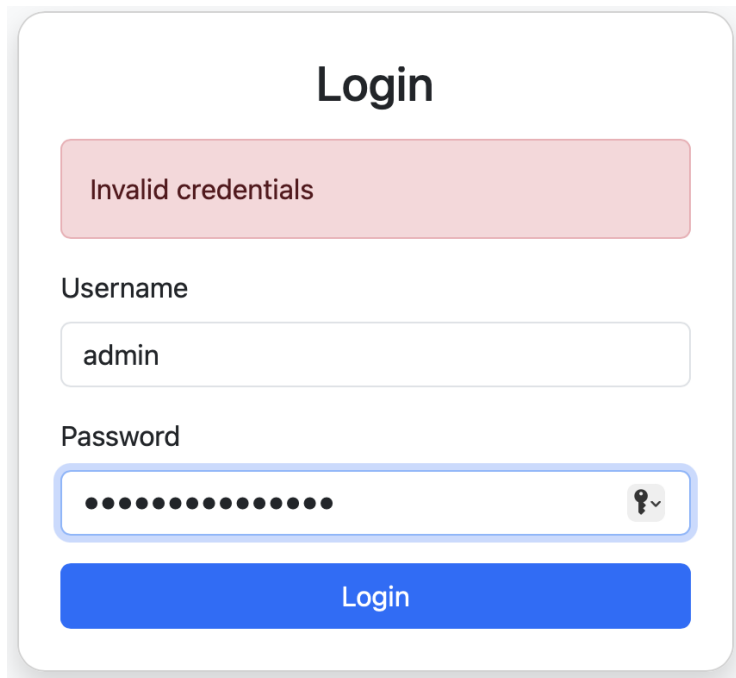
Projektets linux server er bygget op på ZimaBoard 832 modellen, et 3D printet case samt et 250gb Sata SSD-tilkobling som udvidet drevplads. Udover hardware delen er der gjort brug af casaos som underliggende software system. Som nævnt tidligere i grund teori er serveren bygget ud fra et mikrotjeneste perspektiv for at sikre isolering af individuelle programmer og tjenester. Dette resulterer i en højere oppe tid og skaber et mere robust servermiljø. I dette afsnit vil der blive præsenteret resultaterne af opsatte services, funktioner, forbindelser samt interaktioner.

Frontend



Dataen bliver fremvist korrekt på vores frontend i en dynamisk graf. Der kan både fremvises temperatur, tryk og rpm. Alle tre grafer virker og opdaterer dynamisk, som nyt data kommer ind. Siden bliver hostet af

flask som bruger python, hvor der er blevet oprettet html templates der bliver brugt til det statistiske udseende, python styre datapunkterne.



Login

Invalid credentials

Username

admin

Password

••••••••••••••••

Login

Der er på screenshottet indsat en forkert kode og et korrekt username, dette beviser at der faktisk bliver lavet user authentication på hjemmesiden og at der ikke bare kan logge ind med det samme.

RBAC, eller role-based access control på frontend håndterer hvilket adgangsniveau brugere har på hjemmesiden. Begge brugere på hjemmesiden har adgang til tænd og sluk funktionen til ventilationssystemet. Administrator bruger med fuld adgang til graf og healthchecks på hjemmesiden. Almindelig bruger har kun adgang til hjemmesidens grafer der viser data fra ventilationssystemet.

Logging

service_name /questdb + /mosquitto + /flask + /spb_emulator + /fastapi			
/questdb > 2025-11-27T14:22:10.179940Z I i > 2025-11-27T14:22:10.179940Z I i > 2025-11-27T14:22:10.177711Z I i > 2025-11-27T14:22:07.854921Z I p > 2025-11-27T14:22:07.854145Z I i > 2025-11-27T14:22:07.853968Z I i > 2025-11-27T14:22:07.851600Z I p > 2025-11-27T14:22:05.181352Z I i > 2025-11-27T14:22:05.179683Z I i > 2025-11-27T14:22:01.063868Z I h > 2025-11-27T14:22:01.063822Z E i > 2025-11-27T14:22:01.063617Z I h > 2025-11-27T14:22:00.176465Z I i	/fastapi > INFO: 127.0.0.1:47548 - "GE" > INFO: 127.0.0.1:51196 - "GE" > INFO: 127.0.0.1:56928 - "GE" > INFO: 127.0.0.1:49084 - "GE" > INFO: 127.0.0.1:60304 - "GE" > INFO: 127.0.0.1:59272 - "GE" > INFO: 127.0.0.1:34484 - "GE" > INFO: 127.0.0.1:51698 - "GE" > INFO: 172.31.1.7:34592 - "GI" > INFO: 127.0.0.1:42510 - "GE" > INFO: 172.31.1.7:59314 - "GI" > INFO: 127.0.0.1:39464 - "GE"	/mosquitto > 1764253335: Received PUBACK from > 1764253335: Sending PUBACK to o > 1764253335: Sending PUBLISH to i > 1764253335: Received PUBLISH fr > 1764253330: Received PUBACK from > 1764253330: Sending PUBACK to o > 1764253330: Sending PUBLISH to i > 1764253330: Received PUBLISH fr > 1764253325: Received PUBACK from > 1764253325: Sending PUBACK to o > 1764253325: Sending PUBLISH to i > 1764253325: Received PUBLISH fr	/flask > 127.0.0.1 - - [27/Nov/2025 15:2: > 127.0.0.1 - - [27/Nov/2025 15:2: > 127.0.0.1 - - [27/Nov/2025 15:2: > 127.0.0.1 - - [27/Nov/2025 15:2: > 127.0.0.1 - - [27/Nov/2025 15:2: > 127.0.0.1 - - [27/Nov/2025 15:2: > 127.0.0.1 - - [27/Nov/2025 15:2: > 127.0.0.1 - - [27/Nov/2025 15:2: > 10.144.213.202 - - [27/Nov/2025 > 10.144.213.202 - - [27/Nov/2025
/spb_emulator > 2025-11-27 15:22:15,175 INFO Sei > 2025-11-27 15:22:10,174 INFO Sei > 2025-11-27 15:22:05,173 INFO Sei > 2025-11-27 15:22:00,172 INFO Sei > 2025-11-27 15:21:55,172 INFO Sei > 2025-11-27 15:21:50,171 INFO Sei > 2025-11-27 15:21:45,170 INFO Sei > 2025-11-27 15:21:40,169 INFO Sei > 2025-11-27 15:21:35,168 INFO Sei > 2025-11-27 15:21:30,167 INFO Sei > 2025-11-27 15:21:25,166 INFO Sei > 2025-11-27 15:21:20,165 INFO Sei			

Der bliver indhentet logs som bliver displayet på et grafana dashboard. Dette er gjort via grafana loki og promtail. Grafana bliver brugt til at bygge selve dashboardet, grafana loki er til aggregering af logs, Loki har fat i både grafana og i promtail, da den er mellempunktet mellem Grafana og Loki. Promtail bliver brugt til at tilgå de lokale logs i Docker og har forbindelse direkte til docker.sock. Fra Grafana kan samme administrator bruger som nævnt i ovenstående afsnit, downloade logs til fejlfinding.

API

Responses	
Curl	
<pre>curl -X 'GET' \ 'http://casas.local:18081/sensors' \ -H 'accept: application/json'</pre>	
Request URL	
<pre>http://casas.local:18081/sensors</pre>	
Server response	
Code	Details
200	Response body <pre>{ "count": 100, "rows": [{ "temperature": 23.15, "pressure": 2.43, "airflow": 902, "timestamp": "2025-12-02T13:52:59.482018" }, { "temperature": 22.69, "pressure": 2.45, "airflow": 938, "timestamp": "2025-12-02T13:52:54.481196" }, { "temperature": 24.66,</pre>

Det er en API der styrer kommunikationen mellem databasen og projektets frontend, samt håndtere healthchecks og at få den data sendt til vores frontend. API'en har som vist også sin egen gui hvor forbindelsen til questDB tables og healthcheck kan testes.

MQTT

```
1764256440: Received PUBLISH from optilogic-spb-emulator (d0, q1, r0, m1251, 'spBv1.0/plantA/DDATA/test-device', ... (30 bytes))
1764256440: Sending PUBLISH to optilogic-spb-ingestor (d0, q1, r0, m1209, 'spBv1.0/plantA/DDATA/test-device', ... (30 bytes))
1764256440: Sending PUBACK to optilogic-spb-emulator (m1251, rc0)
1764256440: Received PUBACK from optilogic-spb-ingestor (Mid: 1209, RC:0)
1764256442: New connection from 127.0.0.1:42892 on port 1883.
```

Der er implementeret sparkplug standard via konfigurerings af publisher samt broker. Dette kan ses ved at brokeren modtager et topic der følger den valgte struktur, der bliver også sendt en publish acknowledge til at verificere det er blevet published fra emulatoren, til clienten, da der bliver brugt Qos 1, skal der være dokumentation der påviser Qos 1 bliver overholdt. Dette bliver gjort ved alt bliver acknowledged af brokeren, samt understøttet af last will fra edge-devicet så der vides hvornår devicet disconnecter.

```
sk@Casaos:~$ mosquitto_sub -p 1883 -t "spBv1.0/+/DDEATH/#" -v
spBv1.0/plantA/DDEATH/test-device DDEATH
```

```
2025-12-01 10:53:06,484 INFO Starting SPB emulator health server on port 8001
2025-12-01 10:53:06,488 INFO Sent DBIRTH -> spBv1.0/plantA/DBIRTH/test-device
2025-12-01 10:53:06,489 INFO Sent DDATA: temp=24.18 tryk=2.40 rpm=979
```

DBIRTH er implementeret sådan at vores edge-node altid viser at den er kommet op.

DDEATH er sat op så det er den sidste besked der bliver sendt af brokeren, når spb_emulatoren stopper abrupt.

Vi overholder sparkplug standarden ved at have opsat publisheren sådan at man kan se hvor informationen kommer fra, både lokationsmæssigt og nede på edge-device niveau. Det gør vi ved at publish på topic'et "/plantA/DDATA/"lokale"/test-device". I lokale feltet vil der i et produktionsmiljø være refereret til hvor enheden sidder, det er ikke implementeret her da det kun er testet med en enhed.

SERVICE HEALTH

Logged in as admin · auto-refresh every 10s

Live from Docker
 Back to main dashboard

Some services are unhealthy. Affected: doxygen (exited)

Containers

Health, status, restart counts & details

Refresh now

CONTAINER	DOCKER STATUS	HEALTH	RESTARTS	DETAILS
doxygen-web	running	no healthcheck	-	-
doxygen	exited	no healthcheck	-	-
grafana	running	healthy	-	Connecting to localhost:3000 (:::1):3000) remote file exists
spb_emulator	running	healthy	-	-
promtail	running	no healthcheck	-	-
spb_ingestor	running	healthy	-	-
fastapi	running	healthy	-	-
mosquitto	running	healthy	-	-
flask	running	healthy	-	-
pyscripts	running	no healthcheck	-	-
questdb	running	healthy	-	-
loki	running	healthy	-	Connecting to localhost:3100 (:::1):3100) remote file exists
big-bear-portainer	running	no healthcheck	-	-

Docker health is driven by the healthcheck definitions in your [docker-compose.yml](#).

Legend

What the statuses mean

- Healthy – healthcheck succeeding within the configured thresholds.
- Starting – container is in grace period or initial checks.
- Unhealthy – recent healthchecks failed; investigate logs or dependencies.
- No healthcheck – container has no healthcheck in compose.

Tip: add healthcheck: to more services to see them here with proper health status.

Healthchecks

Der er også healthchecks, både med i portainer og på vores frontend flask side. Healthchecks opdaterer live alt efter hvilken status containere har, sådan at siden altid kan bruges til at se den reelle status.

De 5 containere der ikke har et healthcheck er bevidst. Pyscripts er udelukkende til scripts, den er ikke obligatorisk for at systemet fungerer, og promtail har så mange dependencies at et healthcheck ikke kan hjælpe med fejlfinding.

Big-bear-portainer bruges af CasaOS til portainer og har intet med projektet at gøre, derfor er der fravalgt healthcheck på denne container.

Doxygen og doxygen-web er ikke sat op med healthchecks da programmet udelukkende skal bruges til dokumentation og kun køres op en enkelt gang.

Filter...

Tables (1)

- sensor_data
 - Columns
 - device (symbol)
 - temp (double)
 - tryk (double)
 - rpm (long)
 - timestamp (timestamp)
 - Storage details

SQL

```
1 | select * from sensor_data
```

Log [17:21:29 GMT+01:00] ✓ 2,885 rows in 202ms Execute: 6.

device	temp	tryk	rpm	timestamp
symbol	double	double	long	timestamp
device	23.78	2.09	1001	2025-11-27T11:05:57.131796Z
device	22.19	2.18	998	2025-11-27T11:06:02.132624Z
device	23.99	2.15	1154	2025-11-27T11:06:07.133374Z
device	24.42	2.16	1092	2025-11-27T11:06:12.134237Z
device	22.88	2.18	1039	2025-11-27T11:06:17.134910Z
device	22.37	2.03	1066	2025-11-27T11:06:22.135719Z
device	22.02	2.14	1123	2025-11-27T11:06:27.136589Z

Der er blevet oprettet en database i questDB, den er oprettet i gui'en men data bliver sendt over porten 8812 som er postgres wireporten, samme port bliver brugt af api'en. Der er blevet valgt postgres wire som forbindelse både til insertion fra edge-devicet og querying fra api'en, det er valget da det har gjort at det er simplere at arbejde med og strukturen i postgres gør at vi kan bruge simple queries.

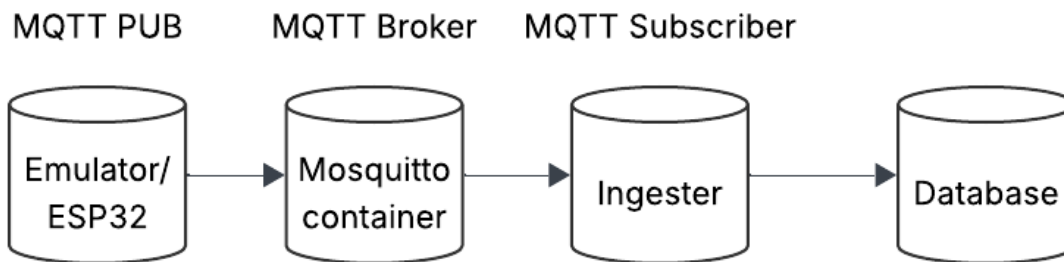
Oprettelsen af en database muliggør effektiv, sikker og skalerbar lagring af data. I dette projekt er der gjort brug af questDB som er baseret på sin egen dialekt af SQL-sproget, bare med GUI interaktion.

Sparkplug-B data kan dog kvalificeres som en semi-struktureret payload og skaber derved problematik for QuestDBs tabelstruktur database. Derfor har det været nødvendigt at gøre brug af parsing via JSON-formatering for at opnå korrekt sparkplug standard samt adaptering i QuestDB.

En essentiel funktion sammensat med den struktureret tabelfunktion i QuestDB er designated timestamp. QuestDB er designet til performanceoptimering igennem timestamps, hvilket ikke som standart findes i traditionelle SQL-databaser. Ved at inkorporerer timestamps, optimerer vi query-hastigheden, mindsker tunge datatransaktioner, vi effektiviserer tidsbaseret forespørgsler og der overholdes retningslinjer for korrekt dataopbevaring. I praksis betyder det, at når data fra en ESP32-enhed eller Sparkplug-B payload emulatoren/ingesteren ender i QuestDB, er det timestampet, der bestemmer alt fra lagringsstrategi til forespørgselsperformance. Et korrekt konfigureret designated timestamp er derfor fundamentet for at questDB kan fungere optimalt som time-series database i system miljøet.

Emulator/Ingestor funktionalitet

Forud for projektet var det planen at bygge, udvikle og integrere hele edge-device enheden med backenden. Men grundet den sene integrering af modbus modulet har vi været nødsaget til at sikre en metode til modtagelse af reel data. Dette blev gjort igennem programmering af to programmer. Ét der kunne emulere vores data og ét der kunne Ingeste den emuleret data ind i databasen. Programmerne skulle kunne følgende: emulere modbus data, følge sparkplug standarden, kunne parse mqtt- sparkplug B data til QuestDB, samt at der var et mål om hurtig integration med ægte sparkplug data fra olimexen, hvis det blev en realitet.



Emulatoren

For at imødekomme sparkplug standarden forsøges der opbygning af payloaden med sparkplug_b_pb2 bibliotek. Men grundet fejl og usikkerhed i dette bibliotek, integrerede vi en fallback version som sikrer en videresendelse af data. De to funktioner hedder *make_spb_payload* og *make_fallback_payload*. Nedenfor ses kodeopbygning.

```
def make_spb_payload(temp, tryk, rpm) -> bytes:
    """ Ægte Sparkplug B payload (hvis sparkplug_b_pb2 findes). """
    p = spb.Payload()
    m = p.metrics.add(); m.name = "temp"; m.double_value = float(temp)
    m = p.metrics.add(); m.name = "tryk"; m.double_value = float(tryk)
    m = p.metrics.add(); m.name = "rpm"; m.long_value = int(rpm)
    return p.SerializeToString()
```

```
def make_fallback_payload(temp, tryk, rpm) -> bytes:
    """ Fallback payload hvis vi ikke har Sparkplug – bare noget læsbart tekst. """
    text = f"temp={temp:.2f},tryk={tryk:.2f},rpm={rpm}"
    return text.encode("utf-8")
```

Efter emulatoren har kørt sine opstartsfunktioner og sikret sig payload struktur, sendes en enkelt TOP_DBIRTH. Herefter oprettes TOP_DDATA med vores payload struktur, samt de randomized tal der indsættes i funktionen.

Ved en u hensigtsmæssig nedlukning af emulatoren vil der blive udført en publish til MQTT-broderen af TOP_DDEATH inden nedlukning for at imødekomme tidligere nævnt sparkplug standard og sikre korrekt logging.

Emulatoren bliver derfor brugt som MQTT-publisher istedet for olimex-esp32. Ved at oprettet to forskellige data muligheder, skabes der en alternativ backup til demonstrering af optimal funktionalitet af prototypens backend. I afsnittet om server funktionalitet, bliver besked typerne forklaret i dybden.

Ingestor

I et tidligere afsnit med QuestDB blev det konkluderet at QuestDB databasen bruger en specifik struktureret tabel dataform. Dette betyder at den semi struktureret sparkplug data som bliver emuleret ikke kan pipes direkte ind i QuestDB. Med denne viden har det været nødvendigt at udvikle et program der kan parse MQTT sparkplug B data ind i den nævnte database.

Dele af koden er opbygget som en MQTT-subscriber. Den skal subscribe direkte på MQTT-broker servicen der kører i en separat container. MQTT-broderen står for at sende data mellem de to klienter. Der bliver brugt det samme bibliotek som der er brugt i emulatoren, dette gør koden mere forståelig da man kun skal forholde sig til ét bibliotek.

```
# -----  
# Environment variables  
# -----  
MQTT_HOST = os.getenv("MQTT_HOST", "mqtt")  
MQTT_PORT = int(os.getenv("MQTT_PORT", "1883"))  
SPB_GROUP = os.getenv("SPB_GROUP", "plantA")  
QDB_ILP_HOST = os.getenv("QDB_ILP_HOST", "questdb")  
QDB_ILP_PORT = int(os.getenv("QDB_ILP_PORT", "8812")) # PostgreSQL wire port  
TABLE = os.getenv("QDB_TABLE", "sensor_data")  
INGESTOR_HEALTH_PORT = int(os.getenv("INGESTOR_HEALTH_PORT", "8002"))  
  
TOPIC_FILTER = f"spBv1.0/{SPB_GROUP}/#"   
  
# ----- PostgreSQL connection -----  
_pg_conn = None  
  
def init_pg_connection():  
    global _pg_conn  
    _pg_conn = psycopg2.connect(  
        host=QDB_ILP_HOST,  
        port=QDB_ILP_PORT,  
        dbname="qdb",  
        user="admin",  
        password="quest"  
    )  
    _pg_conn.autocommit = True  
    logging.info("Connected to QuestDB PostgreSQL wire at %s:%d", QDB_ILP_HOST, QDB_ILP_PORT)
```

Ingestoren er samtidig den service som styrer forbindelsen mellem MQTT-broderen og databasen, den er sat op med de nødvendige variabler til at forbinde til databasen, og gør dette succesfuldt via postgresQL på port 8812.

```
# ----- SIMPLE HTTP HEALTH SERVER -----
class HealthHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        if self.path == "/health":
            self.send_response(200)
            self.send_header("Content-Type", "text/plain")
            self.end_headers()
            self.wfile.write(b"ok")
        else:
            self.send_response(404)
            self.end_headers()

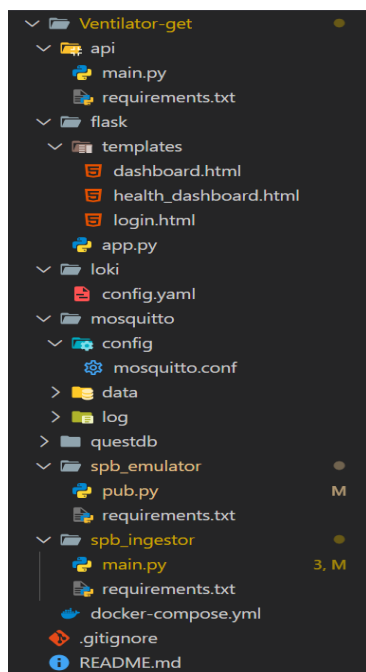
    def log_message(self, format, *args):
        return

def start_health_server():
    server = HTTPServer(("", INGESTOR_HEALTH_PORT), HealthHandler)
    logging.info("Starting ingestor health server on port %d", INGESTOR_HEALTH_PORT)
    server.serve_forever()
```

Der er i koden implementeret en health server der kan melde sin egen health status ud, det gør den ved at serve kode 200 som et OK signal, den vil blive served hver gang den er requested.

Deployment

```
daniel@debian: ~/Venti-pull
daniel@debian:~/Venti-pull$ git clone https://github.com/dbsk67060/Ventilator.git
Cloning into 'Ventilator'...
remote: Enumerating objects: 1231, done.
remote: Counting objects: 100% (25/25), done.
remote: Compressing objects: 100% (17/17), done.
Receiving objects: 33% (407/1231), 94.93 MiB | 9.99 MiB/s
```



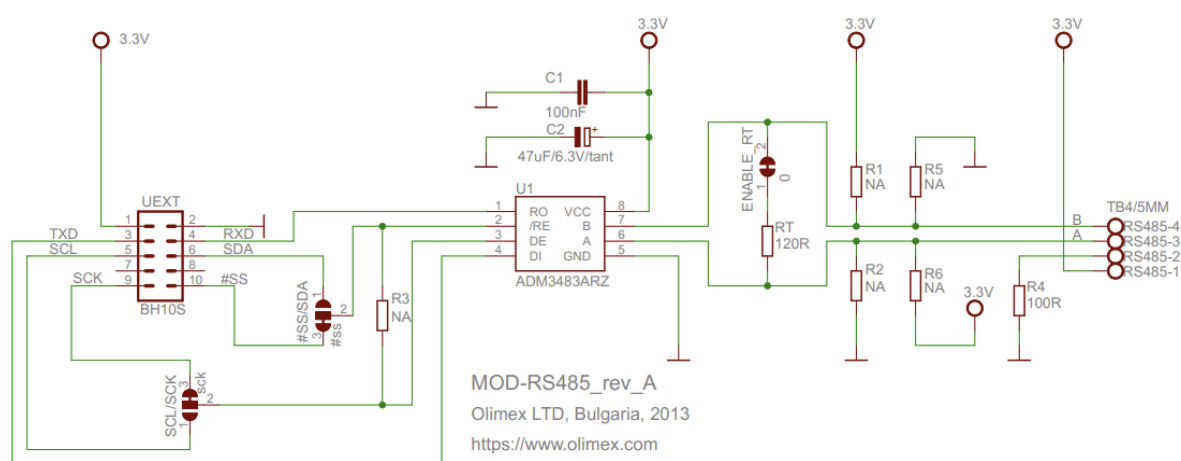
Projektet kan findes i et github repository som kan pulles sådan at den korrekte mappe struktur er til stede samt de tilhørende filer. Det gør det simpelt at få systemet implementeret på en ny server eller andre overflader der har evner til at køre Docker compose.


```
daniel@debian:~/Venti-pull/Ventilator/Ventilator-get$ sudo docker-compose up -d --build
Creating loki      ... done
Creating flask    ... done
Creating mosquito ... done
Creating questdb  ... done
Creating fastapi  ... done
Creating grafana  ... done
Creating spb_emulator ... done
Creating spb_ingestor ... done
Creating promtail ... done
```

Her er det vist at efter man har pullet det, kan man build filerne ved brug af docker-compose servicen. Opsætning af dette bidrager til en nem og effektiv opsætning af systemet i et nyt miljø.

ESP32 – Olimex funktionalitet

ESP32 – Olimex er brugt til oprettelse af forbindelse mellem ventilationssystemet og en IoT-plattform ved brug af modbus RTU UART/RS485-seriel forbindelse som er fremstillet til industriel brug. Modbus RTU sender data som binære pakker og anvender en master/slave funktion hvor ESP32 fungerer som master og sender forespørgsler til ventilationssystemet for nye målinger. ESP32 – Olimex har indbygget strømstyringsfunktioner over Ethernet forbindelse som er velegnet til stabil kontinuerlig drift.



This hardware design by Olimex LTD is licensed under a Creative Commons Attribution-ShareAlike 3.0 Unported License.

Systemopsætningen til brug af ESP32-Olimex via UEXT som vist i ovenstående diagram er opsat efter samme principper og strukturelle model som den konfiguration, der anvendes for emulator og ingestor. Ventilationssystemet registerliste fungerer som oversigt til hvilke registers ventilationssystemet bruger til målinger, setpoints, alarmer og systemstatus. Registerlisten er brugt som udgangspunkt til nedenstående udsnit og ligger i dokumentet som bilag 5.

I ventilatorsystemet er funktioner og data opdelt i adresser. Under adresserne er der registre, her findes der, holding registre, input registre og input status registre.

Holding registre er til at styre systemet, som f.eks. at tænde og slukke systemet, det kan også bruges til manuelt at indstille setpoints.

Input status registret er oftest til alarmer, eller når der opnåes et bestemt setpoint.

Input registre er til at aflæse målinger og data fra systemet.

Opsætningen af ESP32 – Olimex som master gør at man ved brug af *wireHoldingReg(367, 0/3);* kan tænde/slukke for ventilationssystemet. Efterfølgende bliver der læst fra hele blokken som er mellem 10-20 ved brug af

```
// ===== GENERIC MODBUS =====  
bool readRegistersBlock(uint16_t regs[11]) {  
    uint8_t result = modbus.readInputRegisters(10, 11);  
    if (result != modbus.ku8MBSuccess) return false;  
    for (int i = 0; i < 11; i++) {  
        regs[i] = modbus.getResponseBuffer(i);  
    }  
    return true;  
}
```

og gemt i arrays, se bilag 5. Til slut bruges float *getTemperature*, *getPressure* og *getAirFlow* som er *reg 19*, *reg 13* og *reg 15* til at udlæse værdierne fra blokken som sendes til en funktion som omdanner dataen til JSON-format.

ESP32 – Olimex har indbygget TCP/IP forbindelse som er netværksbaseret. Men på baggrund af den længere afprøvningsperiode og tilpasningstid til RTU-seriel forbindelse blev RTU-seriel valgt som forbindelsesmetoden til ventilationssystemet.

MQTT - Olimex

```
rst:0x1 (POWERON_RESET),boot:0x1b (SPI_FAST_FLASH_BOOT)  
config: 0, SPIWP:0xee  
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00  
mode:DIO, clock div:2  
load:0x3fff0030,len:1184  
load:0x40078000,len:13232  
load:0x40080400,len:3028  
entry 0x400805e4
```

```
Starter ventilation (fanStart)  
Ventilation startet: register skriv ok  
Forbinder til WiFi...
```

```
WiFi forbundet. IP: 192.168.3.102  
MQTT: Forsøger at forbinde til broker...  
MQTT: Ikke forbundet, prøver igen...  
MQTT: DBIRTH sendt  
MQTT: Forbundet til broker!  
Sender payload: {"temp":19.9,"tryk":0.0,"rpm":0}  
Payload sendt til MQTT broker.  
Sender payload: {"temp":19.8,"tryk":0.0,"rpm":0}  
Payload sendt til MQTT broker.  
Sender payload: {"temp":19.8,"tryk":0.0,"rpm":0}  
Payload sendt til MQTT broker.  
Sender payload: {"temp":19.8,"tryk":0.0,"rpm":0}  
Payload sendt til MQTT broker.  
Sender payload: {"temp":19.7,"tryk":0.0,"rpm":0}  
Payload sendt til MQTT broker.
```

På ovenstående screenshot ses opstart og forbindelse mellem ESP32 olimexen og MQTT-broderen der kører på Linux serveren. For at forbinde via MQTT fra ESP32 olimexen bliver der gjort brug af PubSubclient biblioteket som tilbyder pre-definerede funktioner til MQTT-forbindelse.

```
const char* MQTT_HOST = "192.168.3.100"; // Mosquitto broker
```

```
const int MQTT_PORT = 1883;
```

Man skal definere IP-adressen for MQTT-broderen samt porten, da porten kan ændres i MQTT konfigurationen.

```
mqtt.setServer(MQTT_HOST, MQTT_PORT);
```

Med funktionen `mqtt.setServer` bliver der defineret hvilken server der skal forbindes til.

```
String TOP_DBIRTH = String("spBv1.0/") + GROUP + "/DBIRTH/" + DEVICE;  
String TOP_DDATA = String("spBv1.0/") + GROUP + "/DDATA/" + DEVICE;  
String TOP_DDEATH = String("spBv1.0/") + GROUP + "/DDEATH/" + DEVICE;
```

Der er også defineret flere topic strings da sparkplug standarden er overholdt og der bliver sendt DBIRTH på initial connection, og DDEATH på disconnection, det bliver gjort over forskellige topics for at separere og nemmere sortere mellem besked typer.

```
String makeJsonPayload(float t, float p, float rpm) {  
    String json = "{";  
    json += "\"temp\":" + String(t, 1) + ",";  
    json += "\"tryk\":" + String(p, 1) + ",";  
    json += "\"rpm\":" + String((int)rpm);  
    json += "}";  
    return json;  
}
```

Der bliver oprettet en payload som bliver sendt via MQTT, den payload er sat op i json struktur da det er et krav i sparkplug standarden, der bliver sendt den samme data som der bliver præsenteret på dashboardet.

```
void mqttReconnect() {  
    Serial.println("MQTT: Forsøger at forbinde til broker...");  
    while (!mqtt.connected()) {  
        Serial.println("MQTT: Ikke forbundet, prøver igen...");  
        if (mqtt.connect("olimex-client", NULL, NULL,  
                        TOP_DDEATH.c_str(), 1, false, "DDEATH")) {  
  
            mqtt.publish(TOP_DBIRTH.c_str(), "DBIRTH", false);  
            Serial.println("MQTT: DBIRTH sendt");  
            Serial.println("MQTT: Forbundet til broker!");  
        }  
    }  
}
```

Der er defineret en funktion til forbindelse af esp32 til broderen, hvor parametrene for at sende enten DBIRTH eller DDEATH er defineret. Når MQTT-forbindelsen fra ESP'en er succesfuld, bliver der sendt en DDEATH (last will and testament) til broderen. Dette gør at broderen gemmer last will and testament samt sender det ud i tilfælde af et pludseligt disconnect. Når forbindelsen er oprettet, bliver der sendt DDEATH (last will and testament) efterfulgt af en publish af DBIRTH.

```
mqtt.publish(TOP_DDATA.c_str(), payload.c_str(), false);
```

Når alt er succesfuldt opsat og har forbindelse, bliver der sendt en `mqtt.publish()`; som har den fulde payload og sender til `topic`'et der håndterer DDATA packets.

Unit-Test resultater

Der er gjort brug af Unittest som løsning til kontrol af logikken, dette gør koden let forståelig og skaber et godt overblik. Testen er udviklet direkte for funktioner relateret til frontenden's processer som f.eks. login, API failure og visning af sensor data. Dette skaber mulighed for at koden bliver gjort produktionsklar og kan debugges samt implementing heraf.

Testen er opbygget med dedikeret testkode, hvor nødvendige libraries er mocket. Det betyder, at de egentlige produktionsbiblioteker ikke indgår, da det ville kræve fuld tilstedeværelse af alle dependencies i testmiljøet. I stedet bliver dependencies som `api`'en mocket, forbindelsen bliver simuleret som en string, hvis koden kan læse stringen, er den godkendt. kan den ikke læse koden, simulere det fejl på forbindelsen, og så er den ikke godkendt. Der bliver brugt samme logik med dependencies i hele koden.

HTML siderne til flask er mocket direkte i test koden. I koden bliver f.eks. "login page" oprettet som en string istedet for en reference til en HTML fil, i produktionsmiljøet vil det være HTML filer, i test er det strings oprettet direkte i koden.

Til flask bliver der gjort brug af unittest, det er vigtigt at frontend logikken er korrekt sådan at end useren samt den it-ansvarlige der hvor systemet bliver integreret har et værktøj til at se både data og Health på containers, så det kan bruge det som udgangspunkt i fejlsøgning. Der bliver kun gjort brug af unittest til Python koden angående frontenden, da det HTML der bliver vist er statiske sider, som bliver læst ind af Python koden.

```
tests/test_flask.py::test_login_page_loads PASSED [ 8%]
tests/test_flask.py::test_login_success_redirects_to_dashboard PASSED [ 16%]
tests/test_flask.py::test_login_fail_shows_error PASSED [ 25%]
tests/test_flask.py::test_dashboard_fetches_sensor_data PASSED [ 33%]
tests/test_flask.py::test_dashboard_handles_api_failure PASSED [ 41%]
tests/test_flask.py::test_dashboard_unauthorized_redirects PASSED [ 50%]
tests/test_flask.py::test_fan_start_stop PASSED [ 58%]
tests/test_flask.py::test_health_returns_ok PASSED [ 66%]
tests/test_flask.py::test_logout_clears_session PASSED [ 75%]
tests/test_flask.py::test_sensor_data_success PASSED [ 83%]
tests/test_flask.py::test_sensor_data_failure PASSED [ 91%]
tests/test_flask.py::test_sensor_data_requires_login PASSED [100%]

===== 12 passed in 0.78s =====
```

I det ovenstående billede kan det ses at alle tests er passed. der bliver testet normale funktioner for siden, at login virker, logout clearer user session, dashboard kan få fat i data, og der vises error ved fejl af login og fetcher den nødvendige mockede data. Der bliver også testet edge cases som forsøg på at få unauthorized adgang til dashboardet og hvis der går dependencies ned, så vil siden stadig være oppe og være funktionel,

selvom det vil være begrænset på baggrund af manglende data eller api failures. Der bliver testet bruger adgang med forskellige logins. Et der kan se healthchecks samt data og et der kun kan se data.

Diskussion

Prototypen er bygget succesfuldt til at kunne udføre en demonstration af interaktion med hjemmeside samt visning af live data, lagring og sikker kommunikation fra database og frontend. Derudover overholdes sparkplug standard helt fra end-to-end.

Prototypen kan videreudvikles ved at tilføje og optimere flere af de brugte services. Prototypen er udviklet med stabilitet og kompatibilitet i fokus, og med mindre fokus på IT-sikkerhed. Prototypen kan aflæse ventilationsdata og overholder de krav som vi har opsat for prototypen.

I dette projekt har vi været igennem flere udfordringer. Det har både testet gruppen og vores tekniske viden. Modbus har medført en stejl indlæringskurve for gruppen, da emnet som industristandard er meget bredt og kræver en vis forståelse, inden det kunne implementeres i projektet. Dette har medført flere timers forsøg og fejlfinding, både på selve modbus delen og modtagelsen/indsættelse af data. Simple ting som holding registers var kontraintuitiv. TCP modbus ville i dette projekt have gjort opstartsfasen nemmere og hurtigere da der ikke skulle tages stilling til noise i kabler, samt pin layout problematikker.

Ud fra logging kravet blev der konstateret manglende erfaring i at arbejde med logs fra andet end linux logs og docker container logs. Der blev brugt lang tid på valg af program og services til at indsamle og vise logs. Der blev forsøgt med graylogs som en start, men dette program var meget tungt og problematisk at implementere. Der blev derfor skiftet til Grafana, Grafana loki og promtail da de var nemmere at implementere og koble sammen.

Perspektivering

ESP32-Olimex kunne bruges med TCP/IP i stedet for seriel forbindelse for at øge skalerbarheden i fremtiden, samt gøre det nemt at integrere på nye lokationer.

Opsætningen af vores Docker pull kan optimeres med et script der flytter filerne til den korrekte lokation på en ny enhed, så dette kan sættes op med en Docker pull fra Git og køre scriptet en enkelt gang.

Ud fra afsnittet om Questdb, kunne en omkodning af Emulator og Ingestor som er skrevet i python, diskuteres af selv samme performanceoptimering. Hvis projektets omfang var udvikling af hurtigste data forbindelse, samt lagring, ville C++ have bedre optimeringsmuligheder, derudover ville der som minimum kunne kigges ind i brugen af ILP-protokollen for hastighedsoptimering. Dog ud fra et service og vedligeholdelses

perspektiv, kunne der argumenteres for en bredere kendskabsgrad, fleksibilitet og hurtig udvikling ved python programmering, samt postgresQL protokollen. Udover det kunne håndteringen af timestamps blive rykket ud til edge-devicet for som tidligere nævnt optimering af hele data pipelinen.

Implementering af Fail2Ban på Zimaboardet samt oprettelse af RBAC ville medføre bedre overblik samt kontrol af adgangen og ændringer lavet på Zimaboardet med tilhørende logs.

TLS/SSL-integration i projektet vil hjælpe med at sikre forbindelsen mellem ESP32 og MQTT. Dette burde i fremtiden indtænkes i projektet inden det bruges i industrielt miljø.

ESP32 Olimex har ESP-IDF native OTA indbygget, hvilket gør det muligt at bruge OTA over en http/https server. Der er også mulighed for brug af FTP, local web-server eller en cloud server til at sende OTA. Dette fungerer ved at ESP'en henter den nye firmware og skriver denne til en inaktiv OTA-partition. Efterfølgende vil ESP'en genstarte og bruge den nye firmware, og vil beholde den gamle partition som fallback mulighed. Andre muligheder er Arduino OTA eller oprettelse af egen OTA server.

Bilag

Bilag nr. 1 - Docker-compose file

```
version: "3.8"

services:
  mqtt:
    image: eclipse-mosquitto:2.0
    container_name: mosquitto
    restart: unless-stopped
    ports:
      - "1883:1883"
    volumes:
      - /DATA/AppData/mosquitto/config:/mosquitto/config
      - /DATA/AppData/mosquitto/data:/mosquitto/data
      - /DATA/AppData/mosquitto/log:/mosquitto/log
    environment:
      - TZ=Europe/Copenhagen
    healthcheck:
      test: ["CMD-SHELL", "mosquitto_pub -h localhost -t healthcheck/test -m 'ping' -q 0 || exit 1"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 10s

  questdb:
    image: questdb/questdb:latest
    container_name: questdb
    restart: unless-stopped
    environment:
      - TZ=Europe/Copenhagen
    ports:
      - "9001:9000"
      - "8812:8812"
    volumes:
      - /DATA/AppData/questdb:/var/lib/questdb
    healthcheck:
      # Simple TCP check on the HTTP port (QuestDB web console)
      test: ["CMD-SHELL", "bash -c 'echo > /dev/tcp/localhost/9000 || exit 1'"]
      interval: 30s
      timeout: 5s
      retries: 3
      start_period: 20s

  fastapi:
    image: tiangolo/uvicorn-gunicorn-fastapi:python3.11
    container_name: fastapi
    restart: unless-stopped
    working_dir: /app
    volumes:
```

```

- /DATA/AppData/api:/app
ports:
- "18081:80"
environment:
- TZ=Europe/Copenhagen
- QDB_HOST=questdb
- QDB_PORT=8812
- MQTT_HOST=mqtt
- MQTT_PORT=1883
- CORS_ORIGIN=http://localhost:3000
- MQTT_SUB_TOPICS=test/#,spBv1.0/#
command: >
sh -c "pip install --no-cache-dir -r requirements.txt &&
    uvicorn main:app --host 0.0.0.0 --port 80"
healthcheck:
# Requires a /health endpoint in your FastAPI app
test: ["CMD-SHELL", "python -c \"import urllib.request; urllib.request.urlopen('http://127.0.0.1/health',
timeout=3)\" || exit 1"]
interval: 10s
timeout: 5s
retries: 3
start_period: 15s

flask:
image: python:3.11-slim
container_name: flask
restart: unless-stopped
working_dir: /app
volumes:
- /DATA/AppData/flask:/app
- /var/run/docker.sock:/var/run/docker.sock:ro
command: >
sh -c "pip install --no-cache-dir flask docker &&
    python app.py"
ports:
- "15000:5000"
environment:
- TZ=Europe/Copenhagen
- QDB_HOST=questdb
- QDB_PORT=8812
- MQTT_HOST=mqtt
- MQTT_PORT=1883
healthcheck:
# Requires a /health endpoint in your Flask app
test: ["CMD-SHELL", "python -c \"import urllib.request;
urllib.request.urlopen('http://127.0.0.1:5000/health', timeout=3)\" || exit 1"]
interval: 10s
timeout: 5s
retries: 3
start_period: 15s

spb_emulator:
image: python:3.11-slim

```



```

container_name: spb_emulator
working_dir: /app
volumes:
  - /DATA/AppData/spb_emulator:/app
command: >
  sh -c "pip install --no-cache-dir -r requirements.txt &&
    python pub.py"
environment:
  - TZ=Europe/Copenhagen
  - MQTT_HOST=mqtt
  - MQTT_PORT=1883
  - SPB_GROUP=plantA
  - SPB_EON=test-eon
  - SPB_DEVICE=test-device
  - SPB_HEALTH_PORT=8001      # <--- matches pub.py health server
depends_on:
  - mqtt
restart: unless-stopped
healthcheck:
  test: ["CMD-SHELL", "python -c \"import urllib.request;
urllib.request.urlopen('http://127.0.0.1:8001/health', timeout=3)\" || exit 1"]
  interval: 10s
  timeout: 3s
  retries: 3
  start_period: 20s

spb_ingestor:
image: python:3.11-slim
container_name: spb_ingestor
working_dir: /app
volumes:
  - /DATA/AppData/spb_ingestor:/app
command: >
  sh -c "pip install --no-cache-dir -r requirements.txt &&
    python main.py"
environment:
  - TZ=Europe/Copenhagen
  - MQTT_HOST=mqtt
  - MQTT_PORT=1883
  - SPB_GROUP=plantA
  - QDB_ILP_HOST=questdb
  - QDB_ILP_PORT=8812      # ILP port inside questdb container
  - QDB_TABLE=sensor_data
  - INGESTOR_HEALTH_PORT=8002 # <--- matches main.py health server
depends_on:
  - mqtt
  - questdb
restart: unless-stopped
healthcheck:
  test: ["CMD-SHELL", "python -c \"import urllib.request;
urllib.request.urlopen('http://127.0.0.1:8002/health', timeout=3)\" || exit 1"]
  interval: 10s
  timeout: 3s

```

```

    retries: 3
    start_period: 20s

# -----
# LOGGING + MONITORING STACK
# -----

loki:
  image: grafana/loki:2.9.7
  container_name: loki
  restart: unless-stopped
  user: "10001"
  ports:
    - "3100:3100"
  volumes:
    - /DATA/AppData/loki/config.yml:/etc/loki/local-config.yml:ro
    - /DATA/AppData/loki/chunks:/loki/chunks
    - /DATA/AppData/loki/rules:/loki/rules

healthcheck:
  test: ["CMD-SHELL", "wget --no-verbose --tries=1 --spider http://localhost:3100/ready || exit 1"]
  interval: 10s
  timeout: 5s
  retries: 5
  start_period: 20s

promtail:
  image: grafana/promtail:2.9.4
  container_name: promtail
  restart: unless-stopped
  user: "10001:986"
  volumes:
    - /var/lib/docker/containers:/var/lib/docker/containers:ro
    - /DATA/AppData/promtail/config.yml:/etc/promtail/config.yml:ro
    - /var/run/docker.sock:/var/run/docker.sock:ro
  command: -config.file=/etc/promtail/config.yml
  depends_on:
    loki:
      condition: service_healthy

# No healthcheck here – promtail’s readiness depends heavily on config & logs present,
# and Docker-level healthchecks tend to flap for it.

grafana:
  image: grafana/grafana:10.4.2
  container_name: grafana
  restart: unless-stopped
  ports:
    - "3000:3000"
  environment:
    - GF_SECURITY_ADMIN_USER=admin

```

```
- GF_SECURITY_ADMIN_PASSWORD=admin
- TZ=Europe/Copenhagen
volumes:
- /DATA/AppData/grafana:/var/lib/grafana
depends_on:
- loki
healthcheck:
test: ["CMD-SHELL", "wget --no-verbose --tries=1 --spider http://localhost:3000/api/health || exit 1"]
interval: 10s
timeout: 5s
retries: 5
start_period: 30s
```

Bilag nr. 2 - SPB_Invester

```
import os
import logging
from typing import Dict, Any
import threading
from http.server import BaseHTTPRequestHandler, HTTPServer
from datetime import datetime

import paho.mqtt.client as mqtt
import pycpg2
import json

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")

# -----
# Sparkplug B protobuf
# -----
try:
    import sparkplug_b_pb2 as spb
    SPB_AVAILABLE = True
    logging.info("sparkplug_b_pb2 imported successfully, SPB decoding enabled.")
except ImportError:
    spb = None
    SPB_AVAILABLE = False
    logging.warning("sparkplug_b_pb2 not found, SPB decoding disabled. Using
JSON/fallback.")

# -----
# Environment variables
# -----
MQTT_HOST = os.getenv("MQTT_HOST", "mqtt")
MQTT_PORT = int(os.getenv("MQTT_PORT", "1883"))
SPB_GROUP = os.getenv("SPB_GROUP", "plantA")
QDB_ILP_HOST = os.getenv("QDB_ILP_HOST", "questdb")
QDB_ILP_PORT = int(os.getenv("QDB_ILP_PORT", "8812"))
TABLE = os.getenv("QDB_TABLE", "sensor_data")
```

```

INGESTOR_HEALTH_PORT = int(os.getenv("INGESTOR_HEALTH_PORT", "8002"))

TOPIC_FILTER = f"spBv1.0/{SPB_GROUP}/#{

# ----- PostgreSQL connection -----
_pg_conn = None

def init_pg_connection():
    global _pg_conn
    _pg_conn = psycopg2.connect(
        host=QDB_ILP_HOST,
        port=QDB_ILP_PORT,
        dbname="qdb",
        user="admin",
        password="quest"
    )
    _pg_conn.autocommit = True
    logging.info("Connected to QuestDB PostgreSQL wire at %s:%d", QDB_ILP_HOST,
QDB_ILP_PORT)

# -----
# Topic parsing
# -----
def parse_topic(topic: str) -> Dict[str, str]:
    p = topic.split("/")
    return {
        "type": p[2] if len(p) > 2 else "",
        "device": p[4] if len(p) > 4 else "",
    }

# -----
# Insert into QuestDB
# -----
def ilp_send(device: str, fields: Dict[str, Any]) -> None:
    global _pg_conn
    if _pg_conn is None:
        init_pg_connection()

    columns = ["device"]
    values = [device]

    for k in ("temp", "tryk", "rpm"):
        if fields.get(k) is not None:
            columns.append(k)
            values.append(fields[k])

    columns.append("timestamp")
    values.append(datetime.utcnow())

```

```

col_str = ",".join(columns)
placeholders = ",".join(["%s"] * len(values))

sql = f"INSERT INTO {TABLE} ({col_str}) VALUES ({placeholders})"

with _pg_conn.cursor() as cur:
    cur.execute(sql, values)

logging.info("WROTE PG: device=%s %s", device, fields)

# -----
# Decode payload (Sparkplug → JSON → fallback)
# -----
def decode_payload(b: bytes) -> Dict[str, Any]:

    # -----
    # 1. Sparkplug B protobuf
    # -----
    if SPB_AVAILABLE:
        try:
            payload = spb.Payload()
            payload.ParseFromString(b)
            out: Dict[str, Any] = {}

            for m in payload.metrics:
                if not m.name:
                    continue
                if m.name in ("temp", "tryk"):
                    if m.HasField("float_value"):
                        out[m.name] = float(m.float_value)
                    elif m.HasField("double_value"):
                        out[m.name] = float(m.double_value)
                    elif m.HasField("int_value"):
                        out[m.name] = float(m.int_value)
                elif m.name == "rpm":
                    if m.HasField("int_value"):
                        out["rpm"] = int(m.int_value)
                    elif m.HasField("long_value"):
                        out["rpm"] = int(m.long_value)

            if out:
                return out

        except Exception:
            pass

    # -----
    # 2. JSON (from ESP32)
    # -----

```

```

text = b.decode("utf-8", errors="ignore").strip()

try:
    j = json.loads(text)
    out = {}
    if "temp" in j: out["temp"] = float(j["temp"])
    if "tryk" in j: out["tryk"] = float(j["tryk"])
    if "rpm" in j: out["rpm"] = int(j["rpm"])
    if out:
        return out
except Exception:
    pass

# -----
# 3. Fallback text "temp=23,tryk=12,rpm=850"
# -----
out: Dict[str, Any] = {}
try:
    for part in text.split(","):
        if "=" not in part:
            continue
        k, v = part.split("=", 1)
        k = k.strip()
        v = v.strip()
        if k in ("temp", "tryk"):
            out[k] = float(v)
        elif k == "rpm":
            out["rpm"] = int(v)
except Exception:
    logging.warning("fallback decode failed: %s", text)

return out

# -----
# MQTT callbacks
# -----
def on_connect(client, userdata, flags, rc):
    if rc == 0:
        logging.info("MQTT connected %s:%d", MQTT_HOST, MQTT_PORT)
        client.subscribe(TOPIC_FILTER, qos=1)
        logging.info("Subscribed: %s", TOPIC_FILTER)
    else:
        logging.error("MQTT connect rc=%s", rc)

def on_message(client, userdata, msg):
    meta = parse_topic(msg.topic)
    if meta["type"] not in ("DBIRTH", "DDATA"):
        return

```

```

try:
    metrics = decode_payload(msg.payload)
    if metrics:
        dev = meta["device"] or "device"
        ilp_send(dev, metrics)
    else:
        logging.debug("No metrics decoded for %s", msg.topic)
except Exception as e:
    logging.warning("Decode/insert error: %s", e)

# -----
# Health server
# -----
class HealthHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        if self.path == "/health":
            self.send_response(200)
            self.send_header("Content-Type", "text/plain")
            self.end_headers()
            self.wfile.write(b"ok")
        else:
            self.send_response(404)
            self.end_headers()

    def log_message(self, format, *args):
        return

def start_health_server():
    server = HTTPServer(("", INGESTOR_HEALTH_PORT), HealthHandler)
    logging.info("Starting ingestor health server on %d", INGESTOR_HEALTH_PORT)
    server.serve_forever()

# -----
# Main
# -----
def main():
    threading.Thread(target=start_health_server, daemon=True).start()

    logging.info("Starting ingestor → QuestDB %s:%d table=%s",
                 QDB_ILP_HOST, QDB_ILP_PORT, TABLE)

    client = mqtt.Client(client_id="optilogic-spb-ingestor", clean_session=True)
    client.on_connect = on_connect
    client.on_message = on_message
    client.connect(MQTT_HOST, MQTT_PORT, keepalive=30)
    client.loop_forever()

if __name__ == "__main__":
    main()

```

Bilag nr. 3 - SPB_Emulator

```

import os
import time
import random
import logging
import threading
from http.server import BaseHTTPRequestHandler, HTTPServer # NEW

import paho.mqtt.client as mqtt

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")

# Try to import Sparkplug B proto if it exists
try:
import sparkplug_b_pb2 as spb # normally generated from sparkplug_b.proto
SPB_AVAILABLE = True
logging.info("sparkplug_b_pb2 imported successfully, SPB encoding enabled.")
except ImportError:
spb = None # type: ignore
SPB_AVAILABLE = False
logging.warning(
"sparkplug_b_pb2 not found, SPB encoding disabled. "
"Will publish simple text payloads instead of real Sparkplug B."
)

# Miljø – match your docker-compose (MQTT_HOST=mqtt)
MQTT_HOST = os.getenv("MQTT_HOST", "mqtt")
MQTT_PORT = int(os.getenv("MQTT_PORT", "1883"))
GROUP = os.getenv("SPB_GROUP", "plantA")
DEVICE = os.getenv("SPB_DEVICE", "test-device")

TOP_DBIRTH = f"spBv1.0/{GROUP}/DBIRTH/{DEVICE}"
TOP_DDATA = f"spBv1.0/{GROUP}/DDATA/{DEVICE}"
TOP_DDEATH = f"spBv1.0/{GROUP}/DDEATH/{DEVICE}" # NY

def make_spb_payload(temp, tryk, rpm) -> bytes:
""" Ægte Sparkplug B payload (hvis sparkplug_b_pb2 findes)."""
p = spb.Payload()
m = p.metrics.add(); m.name = "temp"; m.double_value = float(temp)
m = p.metrics.add(); m.name = "tryk"; m.double_value = float(tryk)
m = p.metrics.add(); m.name = "rpm"; m.long_value = int(rpm)
return p.SerializeToString()

def make_fallback_payload(temp, tryk, rpm) -> bytes:
""" Fallback payload hvis vi ikke har Sparkplug – bare noget læsbart tekst."""
text = f"temp={temp:.2f},tryk={tryk:.2f},rpm={rpm}"
return text.encode("utf-8")

```



```

def make_payload(temp, tryk, rpm) -> bytes:
    """Vælg SPB eller fallback alt efter om sparkplug_b_pb2 findes."""
    if SPB_AVAILABLE:
        return make_spb_payload(temp, tryk, rpm)
    else:
        return make_fallback_payload(temp, tryk, rpm)

def make_death_payload() -> bytes:
    """Payload til DDEATH – meget simpel."""
    if SPB_AVAILABLE:
        p = spb.Payload()
        # Man kan sætte bdSeq og evt. metrics her hvis man vil,
        # men en tom payload er ofte nok til demo/emulator.
        return p.SerializeToString()
    else:
        return b"DDEATH"

# ----- SIMPLE HTTP HEALTH SERVER (NEW) -----

HEALTH_PORT = int(os.getenv("SPB_HEALTH_PORT", "8001")) # match docker-compose

class HealthHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        if self.path == "/health":
            self.send_response(200)
            self.send_header("Content-Type", "text/plain")
            self.end_headers()
            self.wfile.write(b"ok")
        else:
            self.send_response(404)
            self.end_headers()

# Silence default logging to stderr
def log_message(self, format, *args):
    return

def start_health_server():
    server = HTTPServer(("", HEALTH_PORT), HealthHandler)
    logging.info("Starting SPB emulator health server on port %d", HEALTH_PORT)
    server.serve_forever()

def main():
    # Start health server in background thread
    threading.Thread(target=start_health_server, daemon=True).start()

    logging.info(
        "Starter SPB emulator -> MQTT %s:%d group=%s device=%s",
        MQTT_HOST, MQTT_PORT, GROUP, DEVICE,
    )
    if not SPB_AVAILABLE:
        logging.warning(
            "sparkplug_b_pb2 mangler – der sendes IKKE ægte Sparkplug B, "

```

```

"kun simple tekstpayloads."
)

# MQTT klient
c = mqtt.Client(client_id="optilogic-spb-emulator", clean_session=True)

# Sæt Last Will til DDEATH FØR connect
ddeath_payload = make_death_payload()
c.will_set(TOP_DDEATH, ddeath_payload, qos=1, retain=False)

c.connect(MQTT_HOST, MQTT_PORT, keepalive=30)
c.loop_start()

# Send DBIRTH én gang
c.publish(TOP_DBIRTH, make_payload(22.5, 2.2, 1000), qos=1, retain=False)
logging.info("Sent DBIRTH -> %s", TOP_DBIRTH)

try:
while True:
temp = 22.0 + random.random() * 3.0
tryk = 2.0 + random.random() * 0.5
rpm = 900 + int(random.random() * 300)

c.publish(TOP_DDATA, make_payload(temp, tryk, rpm), qos=1, retain=False)
logging.info(
"Sent DDATA: temp=%.2f tryk=%.2f rpm=%d ",
temp, tryk, rpm,
)
time.sleep(5)
except KeyboardInterrupt:
pass
finally:
# Eksplicit DDEATH ved pænt shutdown
logging.info("Sender DDEATH -> %s", TOP_DDEATH)
c.publish(TOP_DDEATH, make_death_payload(), qos=1, retain=False)
# Lille pause så den når at blive sendt
time.sleep(1)
c.loop_stop()
c.disconnect()

if __name__ == "__main__":
main()

```

Bilag nr. 4 – ESP32 Olimex.

```

#include <Arduino.h>
#include <WiFi.h>
#include <PubSubClient.h>
#include <ModbusMaster.h>

// ===== MODBUS / RS485 CONFIG =====
#define RX_PIN 36

```

```

#define TX_PIN 4
#define MAX485_DE 5
#define MAX485_RE_NEG 14
#define BAUD_RATE 9600
#define MODBUS_SLAVE_ID 1

ModbusMaster modbus;

// ===== WiFi + MQTT =====
const char* WIFI_SSID = "FMS";
const char* WIFI_PASS = "FMS12345";

const char* MQTT_HOST = "192.168.3.100"; // Mosquitto broker
const int MQTT_PORT = 1883;

WiFiClient espClient;
PubSubClient mqtt(espClient);

// Sparkplug topics (samme format som emulator)
const char* GROUP = "plantA";
const char* DEVICE = "olimex-device";

String TOP_DBIRTH = String("spBv1.0/") + GROUP + "/DBIRTH/" + DEVICE;
String TOP_DDATA = String("spBv1.0/") + GROUP + "/DDATA/" + DEVICE;
String TOP_DDEATH = String("spBv1.0/") + GROUP + "/DDEATH/" + DEVICE;

// ===== RS485 CONTROL =====
void preTransmission() {
    digitalWrite(MAX485_RE_NEG, HIGH);
    digitalWrite(MAX485_DE, HIGH);
}

void postTransmission() {
    digitalWrite(MAX485_RE_NEG, LOW);
    digitalWrite(MAX485_DE, LOW);
}

// ===== GENERIC MODBUS =====
bool readRegistersBlock(uint16_t regs[11]) {
    uint8_t result = modbus.readInputRegisters(10, 11);
    if (result != modbus.ku8MBSuccess) return false;
    for (int i = 0; i < 11; i++) {
        regs[i] = modbus.getResponseBuffer(i);
    }
    return true;
}

```

```

// Extract values
float getTemperature(uint16_t regs[11]) { return regs[9] / 10.0f; }
float getPressure(uint16_t regs[11])    { return regs[3] / 10.0f; }
float getAirFlow(uint16_t regs[11])     { return regs[5]; }

// ===== SPECIALIZED FUNCTIONS =====
void fanStart() {
    Serial.println("Starter ventilation (fanStart)");
    uint8_t result = modbus.writeSingleRegister(367, 3); // Skriv til holding register 367
    med værdien 0 for sluk og 3 for start.
    if (result == modbus.ku8MBSuccess) {
        Serial.println("Ventilation startet: register skriv ok");
    } else {
        Serial.print("Ventilation start fejlede, modbus fejlkode: ");
        Serial.println(result);
    }
}

// ===== BUILD JSON PAYLOAD =====
String makeJsonPayload(float t, float p, float rpm) {
    String json = "{";
    json += "\"temp\":" + String(t, 1) + ",";
    json += "\"tryk\":" + String(p, 1) + ",";
    json += "\"rpm\":" + String((int)rpm);
    json += "}";
    return json;
}

// ===== MQTT CONNECT =====
void mqttReconnect() {
    Serial.println("MQTT: Forsøger at forbinde til broker...");
    while (!mqtt.connected()) {
        Serial.println("MQTT: Ikke forbundet, prøver igen...");
        if (mqtt.connect("olimex-client", NULL, NULL,
            TOP_DDEATH.c_str(), 1, false, "DDEATH")) {

            mqtt.publish(TOP_DBIRTH.c_str(), "DBIRTH", false);
            Serial.println("MQTT: DBIRTH sendt");
            Serial.println("MQTT: Forbundet til broker!");
        }
        delay(1000);
    }
}

// ===== SETUP =====
void setup() {
    pinMode(MAX485_RE_NEG, OUTPUT);
    pinMode(MAX485_DE, OUTPUT);
    digitalWrite(MAX485_RE_NEG, LOW);
}

```

```

digitalWrite(MAX485_DE, LOW);

Serial.begin(115200);
Serial2.begin(BAUD_RATE, SERIAL_8N1, RX_PIN, TX_PIN);

modbus.begin(MODBUS_SLAVE_ID, Serial2);
modbus.preTransmission(preTransmission);
modbus.postTransmission(postTransmission);

// Start ventilation kort efter Modbus init
fanStart();
delay(300);

WiFi.begin(WIFI_SSID, WIFI_PASS);
Serial.println("Forbinder til WiFi...");
while (WiFi.status() != WL_CONNECTED) {
    Serial.print(".");
    delay(200);
}
Serial.println();
Serial.print("WiFi forbundet. IP: ");
Serial.println(WiFi.localIP());

mqtt.setServer(MQTT_HOST, MQTT_PORT);
}

// ===== LOOP =====
void loop() {
    if (!mqtt.connected()) mqttReconnect();
    mqtt.loop();

    uint16_t regs[11] = {0};
    if (!readRegistersBlock(regs)) {
        delay(2000);
        return;
    }

    float t = getTemperature(regs);
    float p = getPressure(regs);
    float af = getAirFlow(regs);

    String payload = makeJsonPayload(t, p, af);
    Serial.print("Sender payload: ");
    Serial.println(payload);
    mqtt.publish(TOP_DDATA.c_str(), payload.c_str(), false);
    Serial.println("Payload sendt til MQTT broker.");

    delay(2000);
}

```

Bilag nr. 5 – Registerliste ventilationssystem

542	VentActual.Cor_RoomTemp1	R	Input Register	10	Yes	Supply,Extract and Room temperatures	3.0	Room temperature 1
543	VentActual.Cor_RoomTemp2	R	Input Register	11	Yes	Supply,Extract and Room temperatures	3.0	Room temperature 2
544	VentActual.Cor_NeedRunTime	I	Input Register	12	No	Supply,Extract and Room temperatures	3.0	Number of minutes in ongoing support heating/cooling
545	VentActual.Cor_SAFPressure	R	Input Register	13	Yes	SAF/SAF Pressure and Flow	3.0	Supply air fan pressure
546	VentActual.Cor_EAFPressure	R	Input Register	14	Yes	SAF/SAF Pressure and Flow	3.0	Extract air fan pressure
547	VentActual.Cor_SAFAirFlow	R	Input Register	15	Yes	SAF/SAF Pressure and Flow	3.0	Supply air fan flow. Scale factor = 1
548	VentActual.Cor_EAFAirFlow	R	Input Register	16	Yes	SAF/SAF Pressure and Flow	3.0	Extract air fan flow. Scale factor = 1
549	VentActual.Cor_CO2Sensor	R	Input Register	17	Yes	CO2	3.0	CO2 (ppm)
550	VentActual.Cor_DemandRunTime	I	Input Register	18	No	CO2	3.0	Number of minutes support run time CO2
551	VentActual.Cor_FrostProtectionTemp	R	Input Register	19	Yes	Frost protection	3.0	Frost protection temp
552	VentActual.Cor_ExtractAirTemp	R	Input Register	20	Yes	Extract air temp/De-icing exchanger	3.0	Exhaust air temp
572	VentActual.Cor_NeedHeatActive	L	Input Status Register	10	Yes	Supply,Extract and Room temperatures	3.0	Is set if ongoing support heating
573	VentActual.Cor_NeedCoolActive	L	Input Status Register	11	Yes	Supply,Extract and Room temperatures	3.0	Is set if ongoing support cooling
574	VentActual.Cor_DemandCO2Active	L	Input Status Register	12	Yes	CO2	3.0	Is set if ongoing support CO2
575	VentActual.Cor_DeicingActive	L	Input Status Register	13	Yes	Extract air temp/De-icing exchanger	3.0	Is set if ongoing de-icing
576	QDig.DI1	L	Input Status Register	14	Yes	Digital inputs	3.0	Value of DI1
577	QDig.DI2	L	Input Status Register	15	Yes	Digital inputs	3.0	Value of DI2
578	QDig.DI3	L	Input Status Register	16	Yes	Digital inputs	3.0	Value of DI3
579	QDig.DI4	L	Input Status Register	17	Yes	Digital inputs	3.0	Value of DI4
580	QDig.DI5	L	Input Status Register	18	Yes	Digital inputs	3.0	Value of DI5
581	QDig.DI6	L	Input Status Register	19	Yes	Digital inputs	3.0	Value of DI6
582	QDig.DI7	L	Input Status Register	20	Yes	Digital inputs	3.0	Value of DI7

Bilag nr. 6 – FastAPI

```
import os
import time
from datetime import datetime, timedelta

import asyncpg
from fastapi import FastAPI
from fastapi.responses import JSONResponse

# Force UTC inside container
os.environ["TZ"] = "UTC"
time.tzset()

app = FastAPI()

# --- QuestDB config ---
QDB_HOST = os.getenv("QDB_HOST", "172.18.0.4")
QDB_PORT = int(os.getenv("QDB_PORT", "8812"))
QDB_USER = os.getenv("QDB_USER", "admin")
QDB_PASSWORD = os.getenv("QDB_PASSWORD", "quest")
QDB_DATABASE = os.getenv("QDB_DATABASE", "qdb")

async def get_qdb_connection():
    return await asyncpg.connect(
        host=QDB_HOST,
        port=QDB_PORT,
        user=QDB_USER,
        password=QDB_PASSWORD,
        database=QDB_DATABASE,
    )

# -----
# Basic test routes
```

```

# -----
@app.get("/")
def root():
    return {"message": "FastAPI backend for QuestDB"}

@app.get("/health")
async def health():
    try:
        conn = await get_qdb_connection()
        await conn.execute("SELECT 1")
        await conn.close()
        return {"status": "ok", "questdb": "up"}
    except Exception as e:
        return JSONResponse(
            status_code=500,
            content={"status": "error", "questdb": "down", "detail": str(e)},
        )

# -----
# Main data endpoint
# -----
@app.get("/sensors")
async def query_with_asyncpg():
    conn = await get_qdb_connection()
    try:
        rows = await conn.fetch(
            """
            SELECT
                temp      AS temperature,
                hum        AS humidity,
                tryk        AS pressure,
                rpm         AS airflow,
                timestamp
            FROM sensor_data
            ORDER BY timestamp DESC
            LIMIT 100
            """
        )
        data = [dict(row) for row in rows]
        return {"count": len(data), "rows": data}
    finally:
        await conn.close()

# -----
# QuestDB version check
# -----
@app.get("/qdbversion")
async def qdb_version():
    conn = await get_qdb_connection()
    try:
        version = await conn.fetchval("SELECT version()")
    
```

```

        return {"QuestDB_version": version}
    finally:
        await conn.close()

```

Bilag nr. 7 – Flask

```

from flask import Flask, render_template, request, redirect, url_for, session, jsonify
from datetime import datetime
import os
import docker
import requests

ESP32_IP = "http://xxx.xxx.xxx.xxx" # IP adresse på ESP32 - ændre til rigtige

app = Flask(__name__)
app.secret_key = os.getenv("SECRET_KEY", "supersecretkey")

# Docker client
docker_client = docker.DockerClient(base_url="unix://var/run/docker.sock")

# FastAPI URL - ændr hvis IP/hostname er anderledes
FASTAPI_URL = os.getenv("FASTAPI_URL", "http://fastapi:80")

# Dummy user
USERS = {
    "admin": {
        "password": "password",
        "role": "admin"
    },
    "user": {
        "password": "password",
        "role": "user" # almindelig bruger, ingen adgang til health
    }
}

# -----
# LOGIN
# -----
@app.route("/", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        username = request.form.get("username")
        password = request.form.get("password")

        user = USERS.get(username)
        if user and password == user["password"]:

```



```

        session["user"] = username
        session["role"] = user["role"]    # gem rollen i session
        return redirect(url_for("dashboard"))

    return render_template("login.html", error="Invalid credentials")

    return render_template("login.html")
@app.route("/", methods=["GET", "POST"])
#def login():
#    if request.method == "POST":
#        username = request.form.get("username")
#        password = request.form.get("password")
#        if username == USER["username"] and password == USER["password"]:
#            session["user"] = username
#            return redirect(url_for("dashboard"))
#            return render_template("login.html", error="Invalid credentials")
#    return render_template("login.html")

#-----
# TEST - slet når der er testet faerdig
#-----
@app.route("/test-fastapi")
def test_fastapi():
    import requests
    r = requests.get("http://fastapi:80/sensors")
    return r.json()

#-----
# Ventilator start/Stop
#-----
@app.route("/fan/start", methods=["POST"])
def fan_start():
    r = requests.get(f"{ESP32_IP}/fan/start")
    return jsonify({"status": "ok"})

@app.route("/fan/stop", methods=["POST"])
def fan_stop():
    r = requests.get(f"{ESP32_IP}/fan/stop")
    return jsonify({"status": "ok"})

# -----
# DASHBOARD - viser graf
# -----
@app.route("/dashboard")
def dashboard():
    if "user" not in session:
        return redirect(url_for("login"))

    try:
        # Hent data fra FastAPI (som igen henter fra QuestDB)
        r = requests.get(f"{FASTAPI_URL}/sensors", timeout=5)

```

```

        r.raise_for_status()
        data = r.json().get("rows", [])
    except Exception as e:
        data = []
        print(f"Fejl ved hentning af sensordata: {e}")
    return render_template(
        "dashboard.html",
        data=data,
        user=session["user"],
        role=session.get("role")
    )

    # Tilpas felterne her så de matcher din QuestDB-tabel
    labels = [datetime.fromisoformat(row.get("timestamp")).strftime("%d-%m-%Y %H:%M") for
row in data]
    values = [row.get("temperature") for row in data]

    return render_template("dashboard.html", data=data, user=session["user"])

# -----
# LOGOUT
# -----
@app.route("/logout")
def logout():
    session.clear()
    return redirect(url_for("login"))

# -----
# HEALTHCHECK ENDPOINT
# -----
@app.route("/health")
def health():
    return {"status": "ok"}, 200

# -----
# HEALTH DATA API (Docker)
# -----
@app.route("/health-data")
def health_data():
    if "user" not in session:
        return jsonify({"error": "unauthorized"}), 401

    containers_info = []
    for c in docker_client.containers.list(all=True):
        state = c.attrs.get("State", {})
        health = state.get("Health")

        last_log_output = None
        if health and health.get("Log"):

```

```

        last_entry = health["Log"][-1]
        last_log_output = last_entry.get("Output")

        containers_info.append({
            "name": c.name,
            "status": state.get("Status"),
            "health": health.get("Status") if health else None,
            "running": state.get("Running"),
            "restart_count": state.get("RestartCount"),
            "health_log": last_log_output,
        })

    return jsonify(containers_info)

# -----
# NYT ENDPOINT - Flask-API til frontend
# -----
@app.route("/sensor-data")
def sensor_data():
    if "user" not in session:
        return jsonify({"error": "unauthorized"}), 401
    try:
        r = requests.get(f"{FASTAPI_URL}/sensors", timeout=5)
        r.raise_for_status()
        data = r.json()
        return jsonify(data)
    except Exception as e:
        return jsonify({"error": str(e)}), 500

# -----
# HEALTH DASHBOARD
# -----
@app.route("/health-dashboard")
def health_dashboard():
    if "user" not in session:
        return redirect(url_for("login"))
    return render_template("health_dashboard.html", user=session["user"])

# -----
# START APP
# -----
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=True)

```